

# **Lecture 1 (INT 332) BASICS OF DEVOPS**

## **Revolutionizing Containerization**

# Why Devops



# Understanding the concepts of Virtualization and Containerization



LOVELY  
PROFESSIONAL  
UNIVERSITY

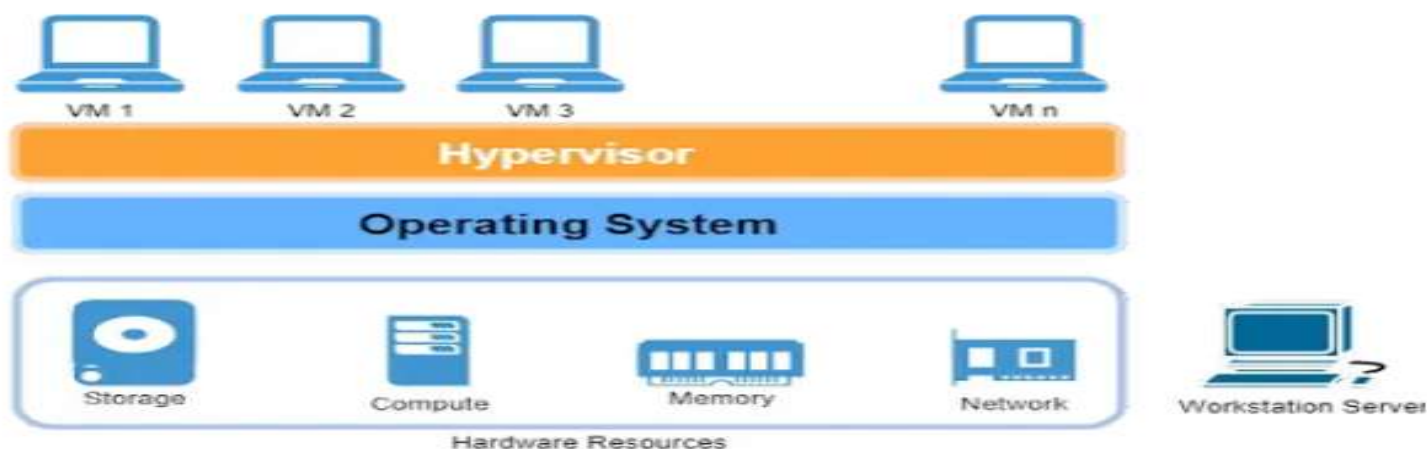


Imagine a peaceful independent house.

## The Apartment Era (Virtualization)



Infrastructure followed the same path.



One physical server was divided into multiple **virtual machines**.  
Each VM felt like its own house — with its own OS, memory, and CPU.

Utilization improved.  
Costs reduced.

# Virtualization



- ◎ With the changing times, businesses started looking for solutions to reduce overhead costs, enhance scalability, and standardize application deployment process.
  
- ◎ They started considering the following two approaches to reduce costs –
  1. Virtualization
  2. Containerization

# What is Virtualization?



- ◎ Virtualization is the process of **partitioning** a physical server into multiple virtual servers.
- ◎ The process of partitioning is carried out using a software called **'hypervisor'**.
- ◎ After partitioning, the virtual servers act and perform just as a physical server.
- ◎ Essentially, it means **using the same hardware** set-up more efficiently, thereby freeing the resources for other tasks or retiring the resources altogether.

# Advantages of Virtualization

- ◎Enhanced performance
- ◎Promotes agile IT infrastructure
- ◎Promotes use of resources in optimum manner
- ◎Better disaster management if any physical hazards take place to resources
- ◎Better security as the infected VM can be isolated from other VMs and the host server
- ◎Space saving
- ◎The capital investment cost on hardware is saved, and thereby maintenance cost is saved, hence overall cost savings for a business.



# **Drawbacks of Virtualization which lead to concept of containerization**



- 1. Heavy Resource Overhead**
- 2. Slow Boot Time and Poor Agility**
- 3. Inefficient Scalability**
- 4. Limited Application Portability**
- 5. Higher Management and Maintenance Complexity**
- 6. Increased Infrastructure Cost**

**These drawbacks directly led to the evolution of containerization, which offers lightweight isolation, faster deployment, better portability, and seamless integration with DevOps and microservices architectures.**

## PG Living (Containerization)



High rent.  
Fast life.  
Limited space.

People started choosing **PGs and hostels**.

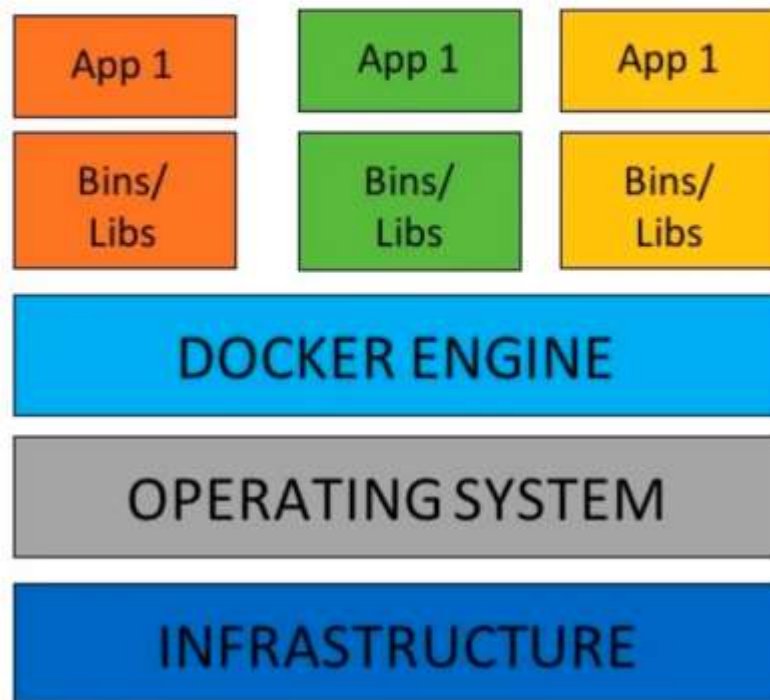
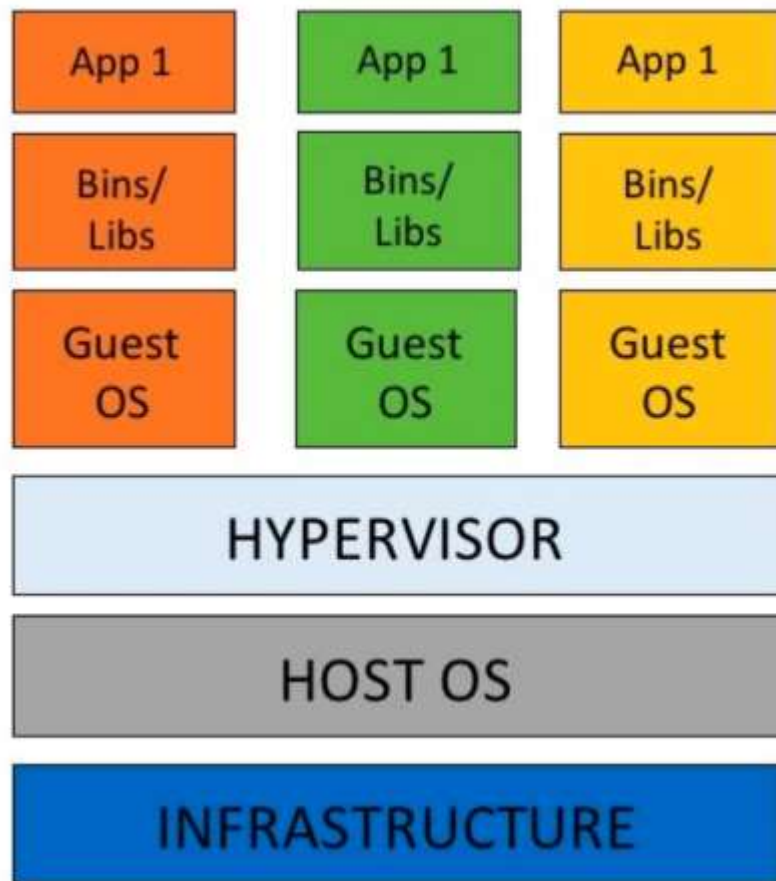
You don't own the building.  
You don't manage electricity.  
You don't worry about maintenance.

# That's exactly what containers do.



- Containers don't carry an entire operating system. They use bare minimum OS.
- They share the host OS and only bring what the application needs.
- They start in seconds. They consume minimal resources.
- You can run dozens — even hundreds — on the same machine.
- Earlier the applications were monolithic , so physical servers were apt,
- later there were components split like backend, frontend, db ,so VM's took care of hosting.
- Now all the applications are turning to microservices, so the containerization tools like docker will serve the purpose.
- Also the portability problem is also solved using the containerization

# Need of containers



# Container Runtime

- A container runtime is the software responsible for creating, starting, stopping, and managing containers on a host system.
- It acts as the execution engine that turns a container image into a running container.

In simple words:

***“Container runtime is what actually runs containers.”***

# Need for Container Runtime

- A container image is just a read-only package (application + dependencies).

To run it, the system needs a runtime to:

- Create an isolated environment
- Apply resource limits
- Start the application process
- Monitor and stop it when required

# Types of Container Runtimes

## 1. High-Level Runtimes

Used by users and DevOps tools.

**Docker Engine**

**containerd**

**CRI-O (Kubernetes-focused)**

**These handle:**

- **Image pulling**
- **Networking**
- **Storage**
- **Container lifecycle**

## 2. Low-Level Runtimes

- Responsible for actually running the container process.
- runc (OCI-compliant)

These directly interact with:

- Linux kernel
- Namespaces
- cgroups

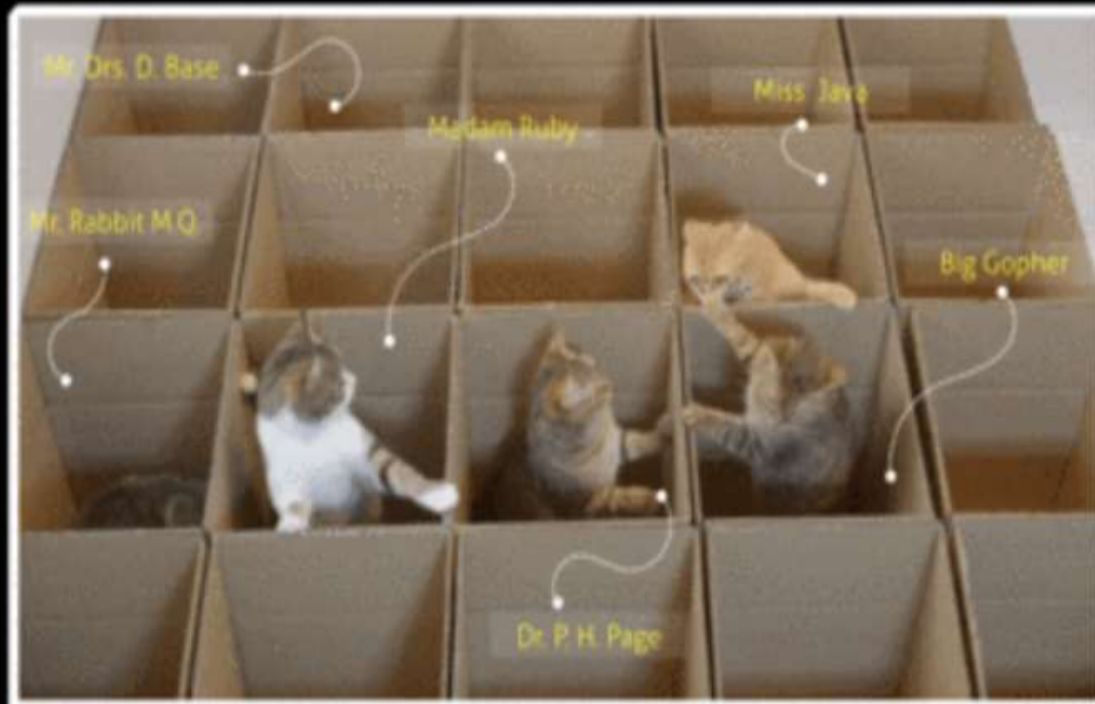


## Real-World Analogy

- **Container Image → Packed lunch box**
- **Container Runtime → Person who opens it, serves food, and cleans up**

# Process Isolation & Namespaces

- **Process isolation ensures that each container runs as if it is alone on the system, even though multiple containers share the same OS kernel.**
- **This isolation is achieved using Linux namespaces.**



containers isolate a process

## Interpretation of Picture

- As shown is this little cat picture, containers are isolated but using networking they still **discover** each other and consume the services if needed.

# Namespaces



- Namespaces are Linux kernel features that partition system resources so that processes see only what belongs to them.
- *“Namespaces create the illusion of a separate system for each container.”*

# Namespaces Used in Containers

## 1. PID Namespace (Process Isolation)

- Each container has its own process tree
- Process IDs inside container start from 1
- A container cannot see host or other container processes

E.g: Inside a container, the main app runs as PID 1, just like init/systemd in a VM.

## 2. Network Namespace



- **Each container gets:**
  - Its own IP address
  - Its own ports
- **Containers can run apps on the same port without conflict**
- **Example:**  
**Multiple Nginx containers all listen on port 80 internally.**

### 3. Mount Namespace

- Each container has its own filesystem view
- File changes inside container do not affect host

### 4. UTS Namespace

- Allows each container to have its own hostname

### 5. User Namespace (Advanced)

- Maps container users to non-root users on host
- Improves security



**Namespaces are like separate cabins in a train:**

- **Same engine (kernel)**
- **Different passengers**
- **No visibility between cabins**

# Control Groups (cgroups) for Resource Limits

Control Groups (cgroups) are Linux kernel features that limit, control, and monitor resource usage of processes.

They ensure that:

- No container can monopolize system resources
- Fair sharing of CPU, memory, and I/O

***“Namespaces isolate, cgroups control.”***

# Need of cgroups

## Without cgroups:

- One container could consume all CPU or RAM
- Host and other containers could crash

# Types of Resources Controlled by cgroups

## 1. CPU

- Limit CPU usage
- Assign CPU shares

## 2. Memory

- Set maximum memory usage
- Kill container if limit exceeded

## 3. Disk I/O

- Limit read/write speeds

## 4. Network (Indirectly)

- Through traffic shaping mechanisms

**cggroups are like electricity meters in apartments:**

- **Each flat has a limit**
- **One tenant cannot use all power**

## Finally for namespaces and cgroups

**Containers are lightweight because they do not virtualize hardware.**

**Instead, a container runtime uses Linux namespaces for process isolation and cgroups for resource control, allowing multiple isolated applications to run efficiently on a shared operating system kernel.**



## Without cgroups (Problem Scenario)

### HOST SYSTEM

CPU: 100%

Memory: 16 GB

#### Container A (No Limits)

Uses **90%** CPU

Uses **14 GB** RAM

Container B  
CRASHED

Container C  
CRASHED

## With cgroups (Controlled)

### HOST SYSTEM

CPU: 100%

Memory: 16 GB

#### Container A (cgroups)

CPU Limit : 30%

RAM Limit: 4 GB

#### Container B

CPU: 30%

RAM: 6 GB

#### Container C

CPU: 40%

RAM: 6 GB

### Kernel Enforces Limits



**cgroups**  
Resource Limits



**Namespaces**  
Isolation

*No Container Can Monopolize Resources*

# Today's Task

**Task 1: A company uses containers for microservices. One faulty container crashes the host due to high memory usage.**

## Questions:

- Which container feature was misconfigured or missing?
- Which kernel mechanism should have prevented this?
- Would namespaces alone solve this issue? Why?



## **Task 2: A server is running three containers:**

- **Container A runs a CPU-intensive task**
- **Container B runs a web server**
- **Container C runs a database**

## **Questions:**

- **Which Linux feature prevents Container A from consuming all CPU?**
- **Which feature ensures Container B cannot see processes of Container C?**
- **Which namespace allows all three containers to use port 80 internally?**

# Container Image

A container image is:

- A read-only blueprint
- Contains:
  - Base OS (minimal)
  - Application code
  - Libraries & dependencies
  - Runtime
  - Configuration

Images are immutable – once built, they do not change.

# Example

**python:3.11-slim** is the image

**This image already includes:**

- **Linux OS**
- **Python 3.11**
- **Standard Python libraries**

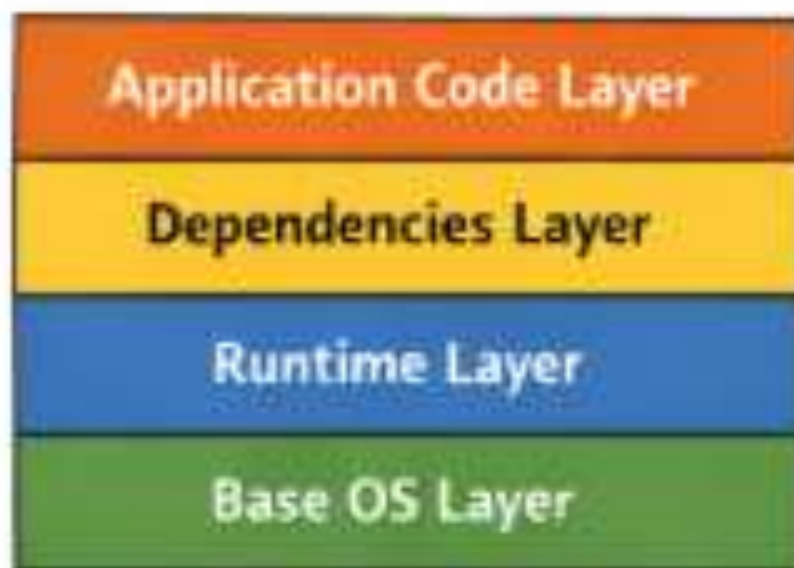
**Each image is built as a stack of layers.**

## **Characteristics of Layers**

- **Each instruction in a Dockerfile = one layer**
- **Layers are:**
  - **Immutable**
  - **Cached**
  - **Reused across images**



## Image Layers



## Faster Builds

- Docker reuses unchanged layers (cache)

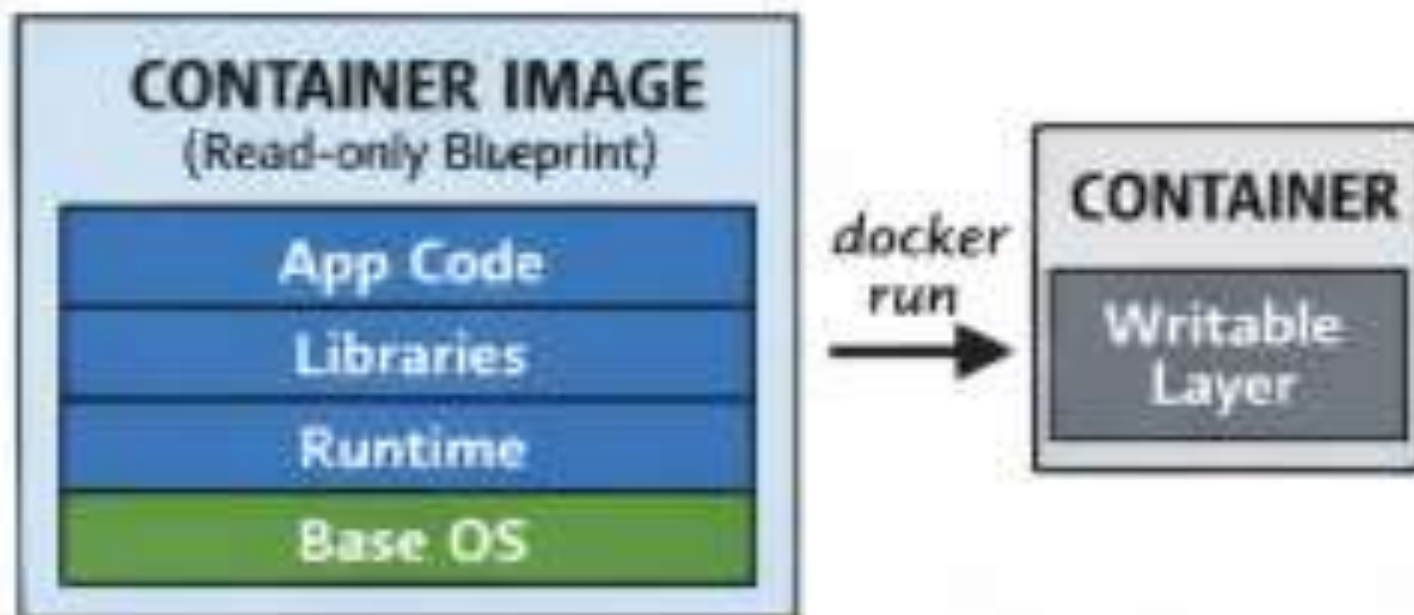
## Smaller Storage

- Same layers shared between images

## Faster Deployment

- Only missing layers are downloaded

## Container Image vs Container



# Image Registry



**An image registry is a centralized repository used to store, manage, version, and distribute container images.**

**It enables:**

- **Sharing images across teams and environments**
- **Consistent deployment in development, testing, and production**
- **Integration with CI/CD pipelines**

**In DevOps, an image registry plays the same role as:**

- **GitHub → for source code**
- **Maven Central → for Java libraries**



# Why Image Registries are Required



## Without a registry:

- Each system must build images independently
- High inconsistency across environments
- Manual software installation
- No version control of application environments

## With a registry:

- Build once, deploy everywhere
- Faster deployments
- Version-controlled environments
- Rollback capability
-

## 3.1 Public Image Registries

**Public registries allow open access to images.**

### **Characteristics:**

- **Images are publicly visible**
- **Anyone can pull images**
- **Often free**
- **Used for base images and open-source applications**

# Private Image Registries



Private registries restrict access using authentication and authorization.

## Characteristics:

- Secure and controlled access
- Used in enterprises
- Stores proprietary applications
- Integrated with IAM and RBAC

## Examples:

- AWS Elastic Container Registry (ECR)
- Azure Container Registry (ACR)
- GitHub Container Registry (GHCR)
- On-premise private registry

# Image Naming Convention

Images follow a standard naming

format:<registry>/<namespace>/<image>:<tag>

Example:docker.io/harpreet/webapp:v1

| Component | Meaning              |
|-----------|----------------------|
| registry  | Location of image    |
| namespace | User or organization |
| image     | Application name     |
| tag       | Version or label     |

# Tags and Versioning

Tags identify different versions of the same image.

## Common Tagging Strategies

| Tag    | Purpose                                 |
|--------|-----------------------------------------|
| latest | Default (not recommended in production) |
| v1, v2 | Versioned releases                      |
| dev    | Development build                       |
| qa     | Testing build                           |
| prod   | Stable production build                 |

# Image Distribution Process

Image distribution refers to how images move from developers to production systems.

## Distribution Flow:

- Developer builds an image locally
- Image is pushed to a registry
- Registry stores the image securely
- Servers pull the image when required
- Containers are created from the image

# Push and Pull Operations

## Push Operation

- Uploads image layers from local system to registry
- Requires authentication
- Only new layers are pushed

## Pull Operation

- Downloads image layers from registry
- Uses caching to avoid re-downloading existing layers
- Faster on repeated deployments

# Role of Image Registries in CI/CD Pipelines

**In modern DevOps pipelines:**

- **Code is committed to Git**
- **CI system builds the container image**
- **Image is pushed to registry**
- **CD system pulls the image**
- **Application is deployed automatically**

**In Actual, Registry acts as the bridge between CI and CD.**





# Comparison Between Public and Private Registry

| Feature  | Public Registry       | Private Registry       |
|----------|-----------------------|------------------------|
| Access   | Open                  | Restricted             |
| Security | Lower                 | High                   |
| Cost     | Mostly free           | Paid                   |
| Use Case | Learning, open source | Enterprise, production |

# QUIZ



**1.What is the primary purpose of an image registry?**

- A. To execute containers**
- B. To store and distribute container images**
- C. To build container images**
- D. To monitor container performance**

**2. Which component of an image name represents the version of the image?**

- A. Registry**
- B. Namespace**
- C. Image name**
- D. Tag**

**3. Which type of image registry is typically used in enterprise environments?**

- A. Public registry**
- B. Open registry**
- C. Private registry**
- D. Anonymous registry**

**4. In CI/CD pipelines, the image registry mainly acts as:**

- A. Source code repository**
- B. Testing environment**
- C. Bridge between CI and CD**
- D. Container runtime**

# Introduction to Docker Hub

## Centralized repository for Docker images

### Features:

- **Public and private repositories**
- **Official images**
- **Automated builds**
- **Importance in DevOps**

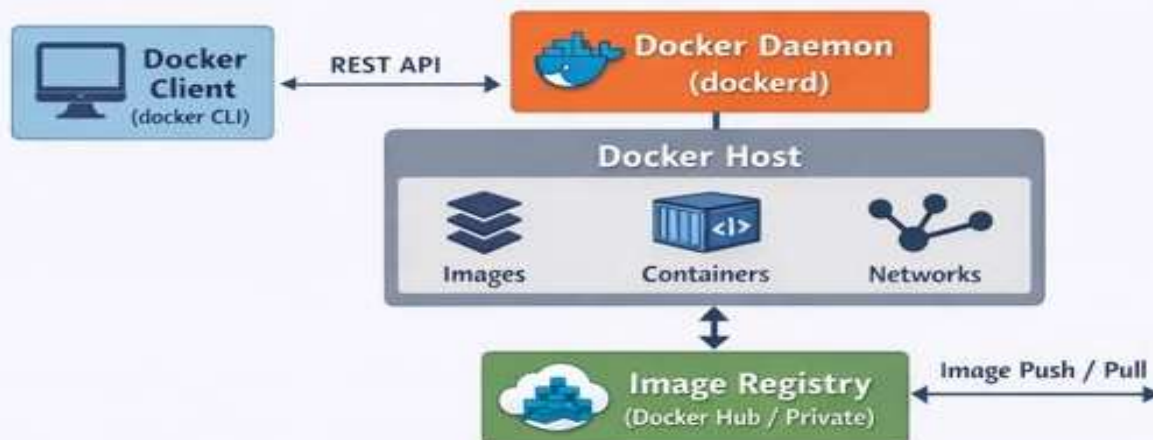
# Docker Architecture

## Key Components:

- - Docker Client
- - Docker Daemon
- - Docker Images
- - Containers
- - Docker Registry (Docker Hub)

# Docker Architecture

## Docker Architecture Overview



## Analogy:



## Example Workflow:



# Docker Life Cycle

## Stages:

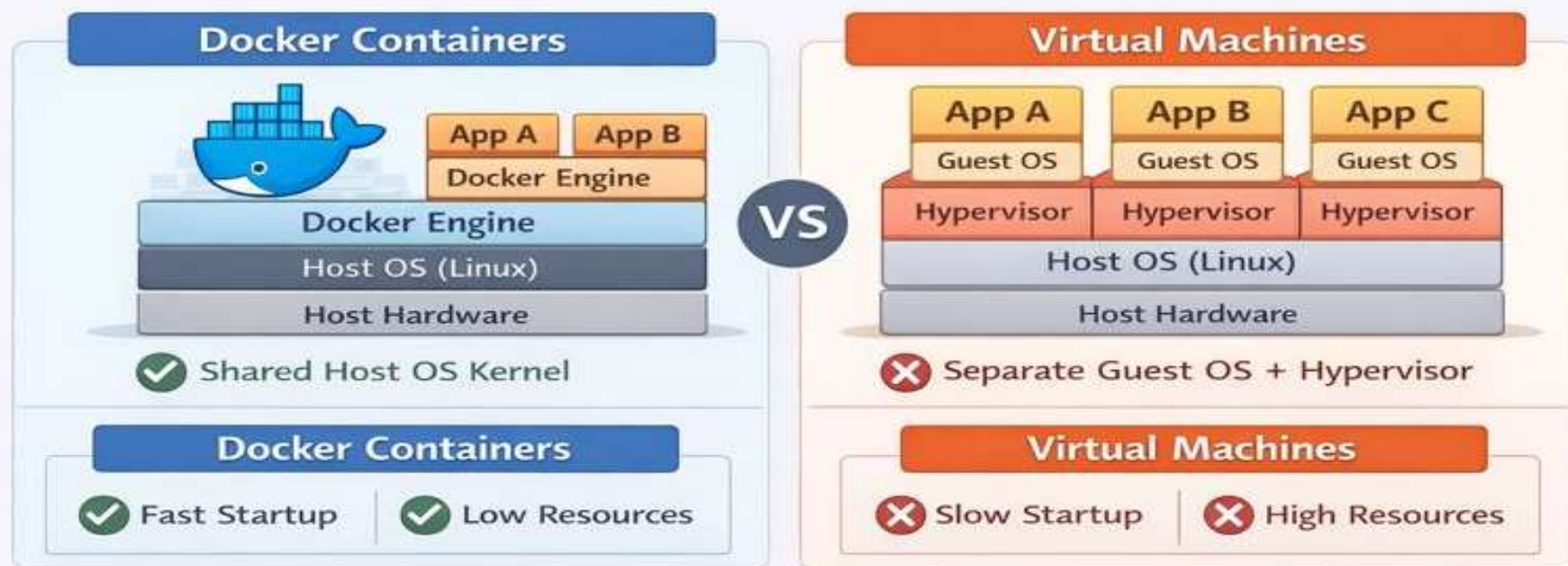
- - Build: Create an image
- - Ship: Share images via Docker Hub
- - Run: Start containers

## Container states:

- - Created
- - Running
- - Paused
- - Stopped

## Why is Docker faster than VM?

- Containers share the Host OS Kernel
- No Guest OS boot needed
- Less resource usage & faster startup





# Conclusion



## Key takeaways:

- Docker simplifies software deployment
- Docker Hub enhances collaboration
- Docker's architecture and lifecycle enable efficient workflows
- Hyper-V integration broadens Docker's capabilities



# **Basic Docker commands**

**Course Code: INT332**

# 1

## Finding the version



One of the first things you want to know is how to find the installed docker version.

- `$ docker --version`
- Docker version 18.09.6, build 481bc77

## 2. Docker info



- Get detailed information about docker installed on the system including the kernel version, number of containers and images, etc.
- **\$ docker info**
  - Containers: 3
  - Running: 1
  - Paused: 0
  - Stopped: 2
  - Images: 3
  - Server Version: 18.09.6
  - Storage Driver: overlay2
  - Backing Filesystem: extfs
  - Supports d\_type: true Native Overlay Diff: true Logging Driver:

# 3. Checking History



- Shows the history of a docker image with the image name mentioned in the command

**\$ docker history httpd**

| <b>IMAGE</b> | <b>CREATED</b> | <b>CREATED BY</b> | <b>SIZE</b> | <b>COMMENT</b> |
|--------------|----------------|-------------------|-------------|----------------|
|--------------|----------------|-------------------|-------------|----------------|

# 4 . Downloading Image



When we need to pull the docker image from dockerhub (docker repository) then ,the following example of pulling the the image and here we are taking Apache HTTP server image.

**\$ docker pull httpd**

```
Using default tag: latest

latest: Pulling from library/httpd

f5d23c7fed46: Pull complete

b083c5fd185b: Pull complete

bf5100a89e78: Pull complete

98f47fcaa52f: Pull complete

622a9dd8cfed: Pull complete

Digest: sha256:8bd76c050761610773b484e411612a31f299dbf7273763103edbdba82acd73642

Status: Downloaded newer image for httpd:latest
```

# 5. Images



To list all the docker images pulled on the system with image details such as TAG/IMAGE ID/SIZE etc.

**\$ docker images**

| REPOSITORY               | TAG    | IMAGE ID     | CREATED      | SIZE   |
|--------------------------|--------|--------------|--------------|--------|
| httpd                    | latest | ee39f68eb241 | 2 days ago   | 154MB  |
| hello-world              | latest | fce289e99eb9 | 6 months ago | 1.84kB |
| sequenceiq/hadoop-docker | 2.7.0  | 789fa0a3b911 | 4 years ago  | 1.76GB |

- The ‘**docker run**’ command is used to create and start a new container from a **Docker** image. The basic syntax of the ‘**docker run**’ command is as follows:
- **docker run [OPTIONS] IMAGE [COMMAND] [ARG...]**
- *where,*
- **OPTIONS:** *optional flags that configure the behaviour of the container, such as setting environment variables, mounting volumes, etc.*
- **IMAGE:** *the name of the Docker image to be run.*
- **COMMAND:** *the command to be executed when the container is started. If not provided, the default command specified in the Docker image will be used.*
- **ARG:** *optional arguments passed to the command.*

**-d means**



# Options Available for executing Docker Run Command



- Some commonly used options for executing the docker run command inside a container:
- *-it: Runs the container in interactive mode, allowing you to interact with the container through the command line.*
- *-d: Runs the container in the background.*
- *-- name: Specifies a name for the container.*
  - rm: Automatically removes the container when it exits.*
- *-p: Maps a host port to a container port.*
- *-e / --env: Sets an environment variable inside the container.*
- *-v: Mounts a host volume inside the container.*

# 1<sup>st</sup> Example : Running a Simple Image:



***docker run httpd echo “Hello, World!”***

- This command runs a new container from the ‘**httpd**’ image and executes the ‘**echo**’ command with the argument “Hello, World!”.

## Running a Container with a Specific Name:

- ***\$ docker run --name my-container httpd echo “Hello, World!”***
- ***--name : this option is to give the name of container***
- ***my-container: it is the name of container***
- ***Echo: it is the command***
- ***“hello,world!” – it is argument used***
- This command runs a new container with the specified name “my-container” from the **httpd** image and executes the echo command with the argument “Hello, World!”.

# What are Environment Variables



Environment variables are **key–value pairs** used by applications to:

- Read configuration
- Store secrets (usernames, passwords)
- Control application behavior **without changing the image**
- *Environment variables allow us to configure containers dynamically.*

# Need of environment variables in Docker



## Without -e:

- Configuration must be **hard-coded** in the image
- Need to rebuild image for every change

## With -e:

- Same image → different behavior
- Secure & flexible configuration
- Widely used in **microservices & cloud deployments**

# 3<sup>rd</sup> Example : Setting Environment:



- **docker run -e MY\_VAR=value httpd env**
- This command sets an environment variable '**MY\_VAR**' to the value "**value**" and runs a new container from the '**httpd**' image.

The '**env**' command is executed to display the list of environment variables in the container.

2<sup>nd</sup> Example: Run a container from Image and display the variable name inside it.

**`docker run -it -e MY_NAME=Harpreet ubuntu bash`**

**It will open the container and you can check the env variable inside it as:**

**Echo \$ MY\_NAME**

**Kindly note :**

**MY\_NAME is created inside the container**

**Host system is not affected**

# Multiple Environment Variables



- `docker run -e APP_ENV=production -e APP_VERSION=1.0 nginx`

Same for MySQL Container

```
docker run -d \  
-e MYSQL_ROOT_PASSWORD=root123 \  
-e MYSQL_DATABASE=college \  
-e MYSQL_USER=admin \  
-e MYSQL_PASSWORD=admin123 \  
mysql:8
```

**Without these variables, MySQL container will fail to start**



# Next is : Passing Environment Variable from Host System



Ex#1:

Step 1: Set variable on host

```
export APP_PORT=8080 (Ubuntu)
```

```
Powershell - $env:APP_PORT=8080
```

```
Check host variable - echo $env:APP_PORT
```

Step 2:

```
docker run -e APP_PORT=$env:APP_PORT nginx env
```

```
docker run -e APP_PORT=$APP_PORT nginx env
```



## Another way : Using --env-file (Clean & Professional Way)

Create .env file

DB\_HOST=localhost

DB\_USER=root

DB\_PASS=secret

Then Run container

`docker run --env-file .env myapp`

# 4th Example



```
$ docker run -it -d httpd
```

Here :

- **-it** flag enables interactive mode, so you can interact with the container's command line
- The **-d** flag runs the container in the background and the **httpd** argument specifies the image name to use for the container. This command will pull the **httpd** image from Docker Hub if it is not already present on your local machine, create a container based on that image, and start it in the background.

# Passing **Environment Variables** to Containers



***Dynamic Configuration at Runtime***

# -p optional flag in run command



- **-p stands for *publish port*.**

It maps a **port on your host machine** to a **port inside the container**.

In simple words:

- **-p allows users outside the container to access an application running inside the container.**

By default:

- Containers run in an **isolated network**
- Applications inside containers are **NOT accessible from the host/browser**
- So if a web server is running **inside a container**, you **must expose it** using -p.

# Syntax of -p



`docker run -p <HOST_PORT>:<CONTAINER_PORT> IMAGE`

| Part                        | Description                          |
|-----------------------------|--------------------------------------|
| <code>HOST_PORT</code>      | Port on your system (Laptop/Server)  |
| <code>CONTAINER_PORT</code> | Port where app runs inside container |

# EX#1:



- Without `-p`
- `docker run nginx`
- Try opening browser: But it is Not accessible  
As no port mapping done here

## **Now try to Run Nginx with -p**

- `docker run -p 8080:80 nginx`

## **What happens internally?**

Browser → localhost:8080 → Docker Host → Container:80 → Nginx



- `docker run -d -p 3307:3306 --name mysql-test -e MYSQL_ROOT_PASSWORD=root123 mysql`

`docker exec -it mysql-test bash`

`mysql -h 127.0.0.1 -P 3306 -u root -p`

# 7 What's running?



- To lists all the docker containers are running with container details.
- .
- **\$ docker ps**

| CONTAINER ID | IMAGE | COMMAND            | CREATED        | STATUS        | PORTS  | NAMES        |
|--------------|-------|--------------------|----------------|---------------|--------|--------------|
| @9ca6feb6efc | httpd | "httpd-foreground" | 36 seconds ago | Up 33 seconds | 80/tcp | suspicious_b |

# TASK #1:



- Run Ubuntu container
- Pass `-e COLLEGE=CSE`
- Show `echo $COLLEGE`
- Stop container then check what happened

# 8. ps -a



To list all the docker containers running/exited/stopped with container details

- **\$ docker ps -a**

| CONTAINER ID | IMAGE                          | COMMAND                  | CREATED        | STATUS                   | PORTS    |
|--------------|--------------------------------|--------------------------|----------------|--------------------------|----------|
| 09ca6feb6efc | httpd                          | "httpd-foreground"       | 51 seconds ago | Up 49 seconds            | 80/tcp   |
| 2f6fb3381078 | sequenceiq/hadoop-docker:2.7.0 | "/etc/bootstrap.sh -d"   | 2 weeks ago    | Exited (137) 9 days ago  |          |
| 9f397feb3a46 | sequenceiq/hadoop-docker:2.7.0 | "/etc/bootstrap.sh -..." | 2 weeks ago    | Exited (255) 2 weeks ago | 2122/tcp |
| 9b6343d3b5a0 | hello-world                    | "/hello"                 | 2 weeks ago    | Exited (0) 2 weeks ago   |          |

# 9. exec



- Access the docker container and run commands inside the container in bash

**\$ docker exec -it 09ca6feb6efc bash/env**

**09ca6feb6efc --- it is Id of the container**

```
root@09ca6feb6efc:/usr/local/apache2# ls

bin  build  cgi-bin  conf  error  htdocs  icons  include  logs

root@09ca6feb6efc:/usr/local/apache2#
```

Type *exit* and press enter to come out of the container.

# 10. Removing container



- Remove the docker container with container id mentioned in the command.

```
$ docker rm 9b6343d3b5a0
```

Run the below command to check if the container got removed or not.

```
$ docker ps -a
```

| CONTAINER ID | IMAGE                          | COMMAND                  | CREATED            | STATUS                   | PORTS    |
|--------------|--------------------------------|--------------------------|--------------------|--------------------------|----------|
| 09ca6feb6efc | httpd                          | "httpd-foreground"       | About a minute ago | Up About a minute        | 80/tcp   |
| 2f6fb3381078 | sequenceiq/hadoop-docker:2.7.0 | "/etc/bootstrap.sh -d"   | 2 weeks ago        | Exited (137) 9 days ago  |          |
| 9f397feb3a46 | sequenceiq/hadoop-docker:2.7.0 | "/etc/bootstrap.sh -..." | 2 weeks ago        | Exited (255) 2 weeks ago | 2122/tcp |

# 11. Removing image



- Remove the docker image with the docker image id mentioned in the command

**\$ docker rmi fce289e99eb9**

```
Untagged: hello-world:latest
```

```
Untagged: hello-world@sha256:41a65640635299bab090f783209c1e3a3f11934cf7756b09cb2f1e02147c6ed8
```

```
Deleted: sha256:fce289e99eb9bca977dae136fbe2a82b6b7d4c372474c9235adc1741675f587e
```

```
Deleted: sha256:af0b15c8625bb1938f1d7b17081031f649fd14e6b233688eea3c5483994a66a3
```

```
geekflare@geekflare:/home/geekflare$
```

- To remove the image forcefully

```
$ docker rmi -f <image id>
```

```
docker rmi -f 92fa43a2ff60503fdf250
```

```
Untagged: httpd:latest
```

```
Untagged:
```

```
httpd@sha256:7765977cf2063fec486b63ddea574faf8fbed285f  
2b17020fa7ef70a4926cdec
```

```
Deleted:
```

```
sha256:92fa43a2ff60503fdf250ef0a16d4170aa2a47c4b5a60d4  
27724283543a8792a
```



# Practice Questions on Basic Docker Commands



- Run a Docker container named “DB-app” based on the “mongodb” image, and expose port 80 on the host to port 8082 on the container?

`docker pull mongo`

`docker run -d --name DB-app -p 80:8082 mongo`

- You are a DevOps engineer in a startup company. The development team asks you to quickly deploy a simple web page that displays the message:

Use the official httpd Docker image.

Run the container so it is accessible on port **8080** of the host machine.

Create or update an HTML file inside the container to display the required message.

Verify the output using a command-line tool.

Properly stop and remove the container after testing.

- `docker pull httpd`
- `docker run -d --name my-apache -p 8080:80 httpd`
- `docker exec -it my-apache bash -c "echo '<h1>Hello from Docker Apache!</h1>' > /usr/local/apache2/htdocs/index.html"`
- `curl http://localhost:8080`
- `docker stop my-apache`
- `docker rm my-apache`

3. How would you use the docker run command with -it, -e, -v, and --name to:

- Set an environment variable APP\_ENV=production.
- Bind a local directory /app/data to /data inside the container.
- Name the container my\_app.



- `docker pull ubuntu`
- `mkdir C:\app\data`
- `docker run -it --name my_app -e APP_ENV=production -v C:\app\data:/data ubuntu /bin/bash`
- `echo $APP_ENV`
- `echo "Hello this is my first program" > /data/test.txt`
- `cat /data/test.txt`
- `echo "Second line added" >> /data/test.txt`
- `cat /data/test.txt`
- `exit`

# Docker container commands:



## 1. Container Lifecycle Management

- **Run a container** (creates and starts it)
  - `docker run -d -p 80:80 --name my_container nginx`
- **Start a stopped container**
  - `docker start <container_id/name>`
- **Stop a running container**
  - `docker stop <container_id/name>`
- **Restart a container**
  - `docker restart <container_id/name>`
- **Pause a container**
  - `docker pause <container_id/name>`

...contd....

- **Unpause a paused container**
- docker unpause <container\_id/name>
- **Kill a container (force stop)**
- docker kill <container\_id/name>
- **Remove a container**
- docker rm <container\_id/name>
- **Remove all stopped containers**
- docker container prune

## 2. Listing and Inspecting Containers



- **List all running containers**
- `docker ps`
- **List all containers (including stopped ones)**
- `docker ps -a`
- **Show detailed info about a container**
- `docker inspect <container_id/name>`
- **Display resource usage stats of a container**
- `docker stats <container_id/name>`
- **View container logs**
- `docker logs <container_id/name>`
- **Follow logs in real-time**
- `docker logs -f <container_id/name>`



# 3. Container Interaction with Host



- **Execute a command inside a running container**
- `docker exec -it <container_id/name> <command>`
- **Attach to a running container's interactive shell**
- `docker attach <container_id/name>`
- **Copy files between container and host**
  - From container to host:
    - `docker cp <container_id>:<container_path> <host_path>`
  - From host to container:
    - `docker cp <host_path> <container_id>:<container_path>`
- E.g:

# Commands for CI with Host



- `docker exec -it Container ID ls /`
- `docker exec -it Container ID /bin/bash`
- `mkdir /data`
- `echo "Hello Docker" > /data/test.txt`
- `exit`
- `docker attach Container ID`
- `docker cp Container ID:/data/test.txt C:\Users\HP\Desktop\`
- `docker cp C:\Users\HP\Desktop\test.txt Container ID`  
`:/data/sample.txt`
- `docker exec -it Container ID ls /data`
- `docker exec -it Container ID cat /data/sample.txt`

# Task



- Inside this same running container, you are required to:
- Create a new directory named /project.
- Inside /project, create a file called report.txt.
- Add the following text inside the file:  
This is our container interaction with our host machine.
- Verify the file content.
- Copy report.txt from the container to your Desktop.
- Create another file named notes.txt in the same /project directory.
- Append the text:
- Verify that both files exist inside /project.



- `docker exec -it  
63de1b6c3391d6a4687e00e375ca9cfe4685e959c2b774deb8f99  
187beaf3224 mkdir /project`
- `docker exec -it  
63de1b6c3391d6a4687e00e375ca9cfe4685e959c2b774deb8f99  
187beaf3224 sh -c "echo 'Docke Lab Assignment' >  
/project/report.txt"`
- `docker exec -it  
63de1b6c3391d6a4687e00e375ca9cfe4685e959c2b774deb8f99  
187beaf3224 sh -c "echo 'Container File Creation Test' >>  
/project/report.txt"`
- `docker exec -it  
63de1b6c3391d6a4687e00e375ca9cfe4685e959c2b774deb8f99  
187beaf3224 cat /project/report.txt`

- `docker cp`  
63de1b6c3391d6a4687e00e375ca9cfe4685e959c2b774deb8f99  
187beaf3224:/project/report.txt C:\Users\HP\Desktop\
- `docker exec -it`  
63de1b6c3391d6a4687e00e375ca9cfe4685e959c2b774deb8f99  
187beaf3224 sh -c "echo 'This is an additional file inside the  
same container.' > /project/notes.txt"
- `docker exec -it`  
63de1b6c3391d6a4687e00e375ca9cfe4685e959c2b774deb8f99  
187beaf3224 ls /project
- `docker exec -it`  
63de1b6c3391d6a4687e00e375ca9cfe4685e959c2b774deb8f99  
187beaf3224 cat /project/notes.txt

# Practice questions:



- 1. You need to start a new container using the nginx image while setting an environment variable `ENV_MODE=production`. Write the docker run command to achieve this.
- 2. You have a running container named `my_app`, but it's not behaving as expected. You want to check its logs to debug any errors. Write the command to view its logs and follow new log entries in real-time.
- 3. A container named `web_server` was stopped. Write the command to:  
a) Start the container again.  
b) Stop the container when needed.

Basic commands used to create,inspect,verify , delete/remove the volumes

1. `docker volume create <volume name>`
2. `docker volume ls`
3. `docker volume inspect mydata`
4. `docker run -d -v <volumename>:<path where to store in image> --name <containername> <imagename>`

- `docker volume create mydata`
- `docker volume ls`
- `docker volume inspect mydata`
- `docker run -dit --name mycontainer -v mydata:/app/data ubuntu`
- `docker exec -it mycontainer ls /app/data`
- `docker inspect mycontainer`
- `docker rm -f mycontainer`
- `docker volume rm mydata`
- `docker volume prune`





- Create a Docker volume named projectdata.
- Verify that the volume has been created successfully.
- Inspect the volume and identify: The mount point (where Docker stores it on the host system).
- Run an Ubuntu container in detached mode:  
Name the container project\_container.
- Attach the volume projectdata inside the container at /app/data.
- Inside the running container: Create a file named report.txt inside /app/data.
- Add the text: This is my first volume.
- Stop and remove the container.
- Run a new Ubuntu container: Name it project\_container\_new.
- Attach the same volume projectdata at /app/data.
- Verify that report.txt still exists and contains the same data.

- `docker volume create projectdata`
- `docker volume ls`
- `docker volume inspect projectdata`
- `docker run -dit --name project_container -v projectdata:/app/data ubuntu`
- `docker exec -it project_container sh -c "echo This is my first volume" > /app/data/report.txt`
- `docker exec -it project_container cat /app/data/report.txt`
- `docker stop project_container`
- `docker rm project_container`
- `docker run -dit --name project_container_new -v projectdata:/app/data ubuntu`
- `docker exec -it project_container_new ls /app/data`
- `docker exec -it project_container_new cat /app/data/report.txt`

# Task 1



You are a DevOps engineer at a university. The web development team has built a simple HTML-based portal and wants it deployed using Docker.

- Use the official httpd image.
- Name the container college\_portal.
- Run it in detached mode.
- Expose container port 80 to host port 8080.
- Set environment variable ENV=production.
- Attach a Docker volume named portaldata to store website files.
- Modify the index.html file inside the container.
- Verify the output in the browser.
- Check logs if the page is not loading.
- Stop and remove the container.
- Remove the volume and image after testing.

- `docker pull httpd`
- `docker volume create portaldata`
- `docker run -dit --name college_portal -p 8080:80 -e ENV=production -v portaldata:/usr/local/apache2/htdocs httpd`
- `docker exec -it college_portal bash -c "echo '<h1>Welcome to University Portal - Production Mode</h1>' > /usr/local/apache2/htdocs/index.html"`
- `curl http://localhost:8080`
- `docker logs college_portal`
- `docker logs -f college_portal`
- `docker stop college_portal`
- `docker rm college_portal`
- `docker volume rm portaldata`
- `docker rmi httpd`

# Task 2



- Your backend team reports that their MySQL container is failing to start.
- Use mysql:8 image.
- Set required environment variables:
  - MYSQL\_ROOT\_PASSWORD
  - MYSQL\_DATABASE
  - MYSQL\_USER
  - MYSQL\_PASSWORD
- Run in background.
- Expose port 3306 to host port 3307.
- Check logs to identify errors.
- Enter inside container to verify database.
- Stop and remove container after debugging.



- `docker pull mysql:8`
- `docker run -d --name mysql_debug -p 3307:3306 -e MYSQL_ROOT_PASSWORD=root123 -e MYSQL_DATABASE=college -e MYSQL_USER=admin -e MYSQL_PASSWORD=admin123 mysql:8`
- `docker ps`
- `docker logs mysql_debug`
- `docker logs -f mysql_debug`
- `docker exec -it mysql_debug bash`
- `mysql -u root -p // # (Enter password: root123)`
- `exit`
- `Exit`
- `docker stop mysql_debug`
- `docker rm mysql_debug`



**Image creation in detail**



- `mkdir nginx-app`
- `cd nginx-app`
- `notepad index.html`



- `<!DOCTYPE html>`
- `<html>`
- `<head>`
- `<title>Medium Level Docker App</title>`
- `</head>`
- `<body>`
- `<h1>Welcome to My Custom Docker Nginx App</h1>`
- `<p>This is a medium-level Docker project.</p>`
- `</body>`
- `</html>`

- notepad default.conf

```
server {  
    listen 80;  
    server_name localhost;  
  
    location / {  
        root /usr/share/nginx/html;  
        index index.html;  
    }  
}
```

- notepad Dockerfile

FROM nginx:alpine

COPY index.html /usr/share/nginx/html/index.html

COPY default.conf /etc/nginx/conf.d/default.conf

EXPOSE 80

- Dir
- `docker build -t custom-nginx:v1 .`
- `docker run -d -p 8080:80 --name nginx-container custom-nginx:v1`
- `http://localhost:8080`



- `mkdir node-app`
- `cd node-app`
  
- `apt update`
- `apt install nano -y`

- Create app.js

```
const express = require("express");  
const app = express();
```

```
app.get("/", (req, res) => {  
  res.send("Docke Node App Running!");  
});
```

```
app.listen(3000, "0.0.0.0", () => {  
  console.log("Server running on port 3000");  
});
```

- Create package.json

```
{
  "name": "node-docker-app",
  "version": "1.0.0",
  "main": "app.js",
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

- Create Dockerfile

FROM node:18-alpine

WORKDIR /app

COPY package.json .

RUN npm install

COPY app.js .

EXPOSE 3000

CMD ["node", "app.js"]



# Verify files



- Dir

app.js

package.json

Dockerfile- **Rename-Item Dockerfile.txt Dockerfile**

# Build & run & verify



- `docker build -t node-demo:v1 .`
- `Docker run -d -p 3000:3000 --name node-container node-demo:v1`
- `docker ps`
- OPEN BROWSER

<http://localhost:3000>

Docker Node App Running!

- Docker provides different network drivers to control how containers communicate.

## **A. Bridge Network (Default)**

- Default network when you run a container.
- Containers on same bridge can communicate via IP.
- Best for standalone containers on single host.



# List networks

```
docker network ls
```

# Inspect bridge network

```
docker network inspect bridge
```

# Create custom bridge network

```
docker network create my_bridge_net
```

# Run container in custom bridge

```
docker run -d --name container1 --network my_bridge_net nginx
```

```
docker run -d --name container2 --network my_bridge_net alpine sleep 300
```

# Test communication

```
docker exec -it container2 ping container1
```

- Container shares host's networking stack.
- No separate IP.
- Faster performance.
- Linux only.

```
docker run -d --network host nginx  
curl http://localhost
```

- Used in Docker Swarm.
- Connects containers across multiple hosts.

# Initialize swarm

```
docker swarm init
```

# Create overlay network

```
docker network create -d overlay my_overlay
```

# Deploy service using overlay

```
docker service create --name web --network my_overlay -p 8080:80 nginx
```



- Docker has built-in DNS.
- Containers resolve each other by container name (if same custom network).
- Works only in user-defined bridge networks.

- `docker network create app_net`
- `docker run -d --name backend --network app_net nginx`
- `docker run -it --name client --network app_net alpine sh`
- `ping backend`



- Before custom networks, Docker used `--link`.
- Deprecated method. Use networks instead.

```
docker run -d --name db nginx
```

```
docker run -it --name app --link db:database alpine sh
```

# Create volume

```
docker volume create my_volume
```

# List volumes

```
docker volume ls
```

# Run container with volume

```
docker run -d \
```

```
--name my_nginx \
```

```
-v my_volume:/usr/share/nginx/html \
```

```
nginx
```



- `docker volume inspect my_volume`
- `docker volume rm my_volume`

- Mounts host directory into container
- Useful for development

```
docker run -d \  
-v $(pwd)/html:/usr/share/nginx/html \  
-p 8080:80 \  
nginx
```

# Volume Vs Bind Mount

---



# Complete Example



- `docker network create app_bridge`
- `docker network ls`
- `docker volume create mysql_data`
- `docker volume inspect mysql_data`
- `docker run -d --name mysql_db --network app_bridge -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=myapp -v mysql_data:/var/lib/mysql mysql:8`
- `docker run -d --name backend_app --network app_bridge -e DB_HOST=mysql_db node:18-alpine sh -c "apk add curl && sleep 300"`

- `docker exec -it backend_app ping mysql_db`
- `docker run -d --name frontend --network app_bridge -p 8080:80 nginx`

# Microservices and containers



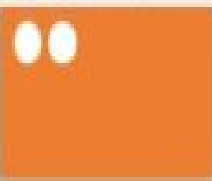
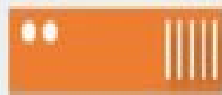
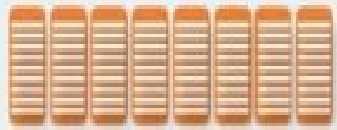
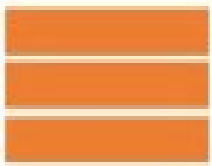
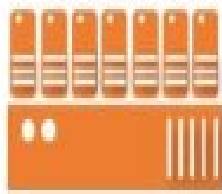
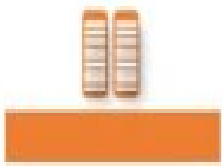
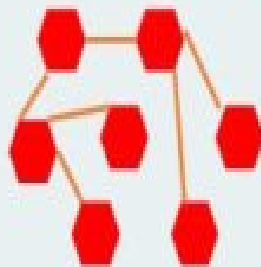
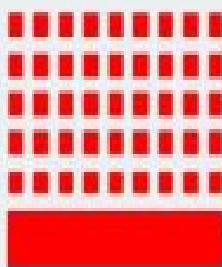
- **Monolithic and Microservices**
- **There has been a lot of change in the development & deployment since the '80s till now. We can trace the evolution of how organizations have developed, deployed, and managed applications over three distinct phases:**

## Development Process

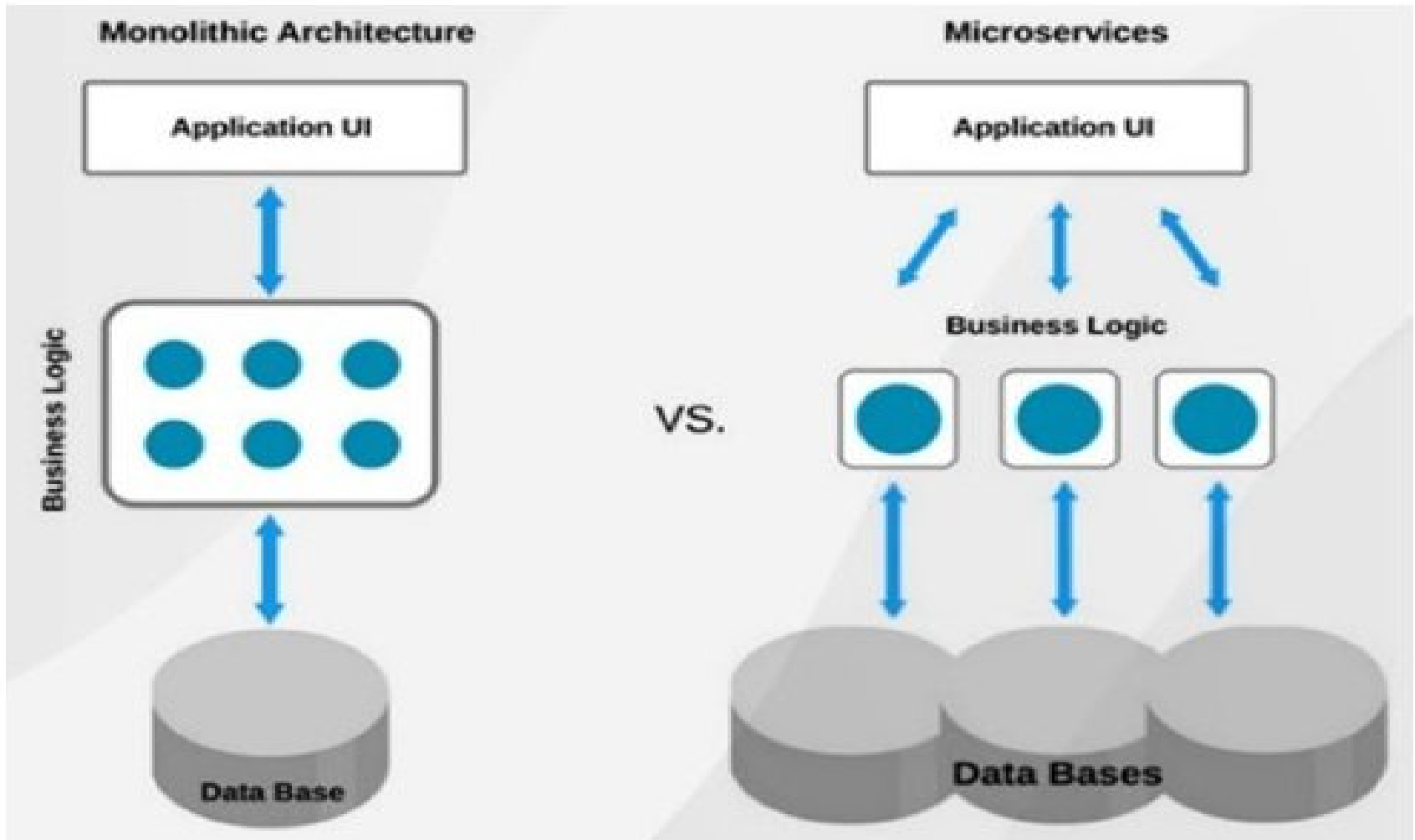
## Application Architecture

## Deployment and Packaging

## Application Infrastructure

|        |                                                                                                       |                                                                                                            |                                                                                                               |                                                                                                          |
|--------|-------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| ~ 1980 | <b>Waterfall</b><br> | <b>Monolithic</b><br>     | <b>Physical Server</b><br> | <b>Datacenter</b><br> |
| ~ 1990 |                                                                                                       |                                                                                                            |                                                                                                               |                                                                                                          |
| ~ 2000 | <b>Agile</b><br>     | <b>N-Tier</b><br>         | <b>Virtual Servers</b><br> | <b>Hosted</b><br>     |
| ~ 2010 | <b>DevOps</b><br>   | <b>Microservices</b><br> | <b>Containers</b><br>     | <b>Cloud</b><br>     |
| Now    |                                                                                                       |                                                                                                            |                                                                                                               |                                                                                                          |

- **80's – mid 90's: Data Centers with physical servers, running monolithic applications, developed using traditional waterfall methodologies.**
- **Late 90s–00s: Datacenters and hosting providers running Unix/Linux servers, the emergence of virtualization (VMWare, KVM), running n-tier applications, developed using agile methodologies.**
- **~ 2010 – Now: Cloud increasingly replacing data centers and hosting, decomposition of applications into microservices, running on a container infrastructure, managed and orchestrated by Kubernetes, developers, and operations collaborating using DevOps methodologies.**



# Monolithic Application

- A monolithic application is constructed as one unit which means it's composed all in one piece. The Monolithic application describes a one-tiered software application within which different components combined into one program from a single platform. The three components are the user interface, the data access layer, and the data store.
- The user interface is the entry point of the application and is also the point of the program that a user will interact with.
- Data access layer is the layer of the program which will wrap the data stored.
- The data store is the most fundamental part of the system and is liable for storing data and retrieving it.

# Example of a Monolithic Application



*An e-commerce website like early versions of Amazon or Flipkart (before adopting microservices).*

- The entire system — user interface, business logic, and database access — was built as one single codebase and deployed together.
- If developers needed to update the “Add to Cart” feature, they had to rebuild and redeploy the whole application.

•

•

•

# 1. User Interface (UI) Example



- **Example:** The web pages or app screens that customers use to browse products, view prices, and click “Buy Now.”
- This part displays information to users and collects their input.
- Technologies (example): HTML, CSS, JavaScript (front-end built into the same application).

## 2. Data Access Layer Example



- **Example: A set of functions or modules in the application that handle communication between the UI and the database.**
  - **For instance, a ProductDAO class (Data Access Object) retrieves product details from the database.**
- **Technologies (example): Java JDBC, Python's SQLAlchemy, or .NET Entity Framework within the same codebase.**



### 3. Data Store Example



- **Example: A relational database like MySQL or PostgreSQL that stores all data — user accounts, product details, and orders.**
- **The application connects directly to this single database to read and write data.**

# Advantages of Monolithic:

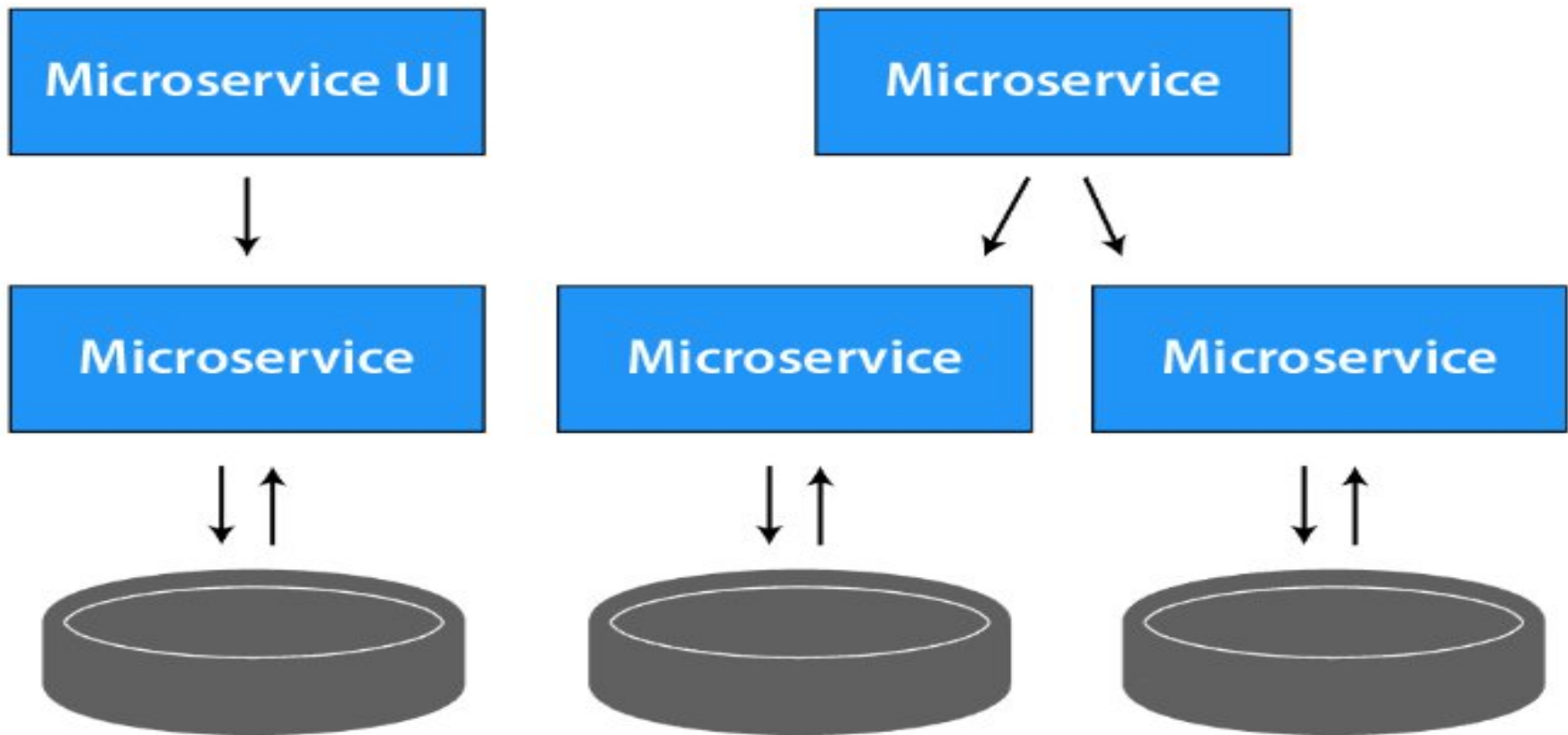


- **Simple to deploy** – In monolithic applications, we don't need to handle many deployments just one file or directory.
- **Easier debugging and testing** – monolithic applications are much easier to debug and test. Since a monolithic app is a single indivisible unit, we can run end-to-end testing much faster.
- **Performance:** Components in a monolith typically share memory which is Quicker than service-to-service communications.

# Disadvantages of Monolithic:



- **New technology barriers** – It's extremely problematic to use new technology in a monolithic application because then the whole application must be rewritten.
- **Scalability**- You can't scale components independently, only the whole application.
- **Size** – It becomes overlarge in size with time and becomes difficult to manage.
- **Difficult to understand** – For any new developer joining the project, it's very difficult to grasp the logic of an oversized monolithic application whether or not his responsibility is related to one functionality.



**Micro-services Architecture**



- a) The idea of microservices is to separate our application into smaller sets and interconnected services rather than building one monolithic application.**
- b) Each module supports a selected business goal and uses a simple and well-defined interface to communicate with other sets of services.**
- c) Each microservice has its own database. Having a database per service is crucial if you wish to learn from microservices because it ensures loose coupling.**

- **Example: *Modern Amazon or Netflix***
  - Instead of one large application, they're made up of many small services, such as:
    - User Management Service
    - Product Catalog Service
    - Payment Service
    - Order Tracking Service
    - Recommendation Service
  - Each service runs independently but communicates with others using APIs.

## Example:

- In an online shopping platform, separate services are built for:
  - User Service → handles login, signup, profile.
  - Order Service → manages order placement and history.
  - Payment Service → processes payments and refunds.
- These services communicate using REST APIs or message queues.

# Each Module Supports a Business Goal



- **Example:**
  - The Payment Service has one clear goal: handle all payment transactions.
  - The Inventory Service focuses only on updating and managing stock levels.
  - Each module (microservice) does one thing well and can be developed, deployed, and scaled independently.
- **Communication Example:**
  - When a user places an order, the Order Service calls the Payment Service via an API like:
  - POST /payment/process
  - The Payment Service confirms the transaction and notifies the Order Service to proceed.



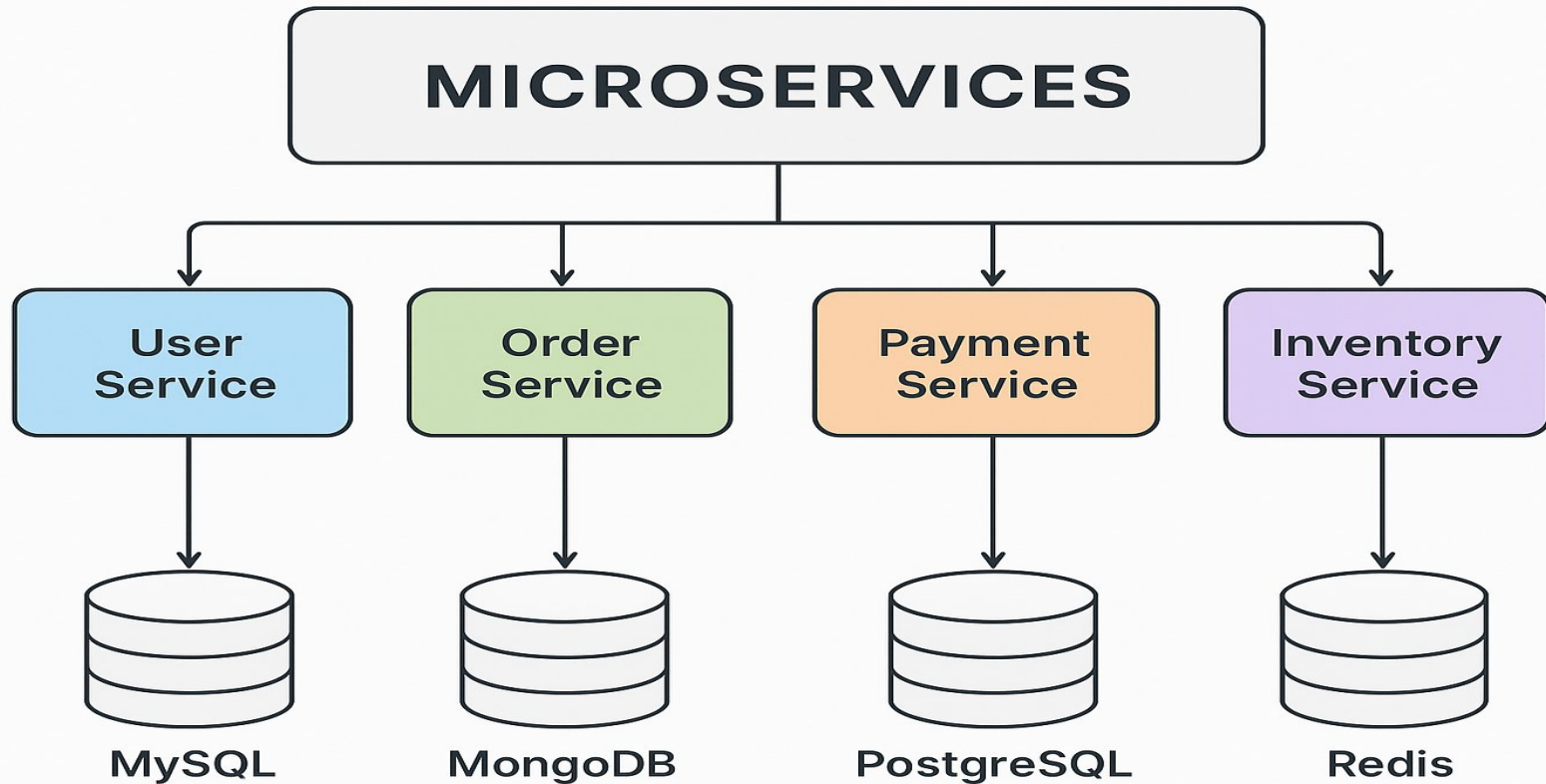
# Each Microservice Has Its Own Database



- **Example:**
  - **User Service Database (MySQL):** Stores usernames, emails, passwords.
  - **Order Service Database (MongoDB):** Stores order details, product IDs, delivery status.
  - **Payment Service Database (PostgreSQL):** Stores payment transactions and invoices.
- **This ensures loose coupling — one service can change its database or schema without affecting others.**

## Other Example :

# Online Food Delivery App (Microservices Example)



# Advantages of Microservices:



| Advantage                 | Explanation                                                                       | Example                                                                                                           | Benefit                                            |
|---------------------------|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|
| 1. Scalability            | Each microservice can be scaled independently based on demand.                    | During festive sales, only the <b>Order Service</b> is scaled up while others remain constant.                    | Efficient resource use and better performance.     |
| 2. Independent Deployment | Teams can deploy or update one microservice without redeploying the whole system. | <b>Payment Service</b> is updated for new payment gateway without affecting <b>User</b> or <b>Order</b> services. | Faster updates and reduced downtime.               |
| 3. Technology Flexibility | Each service can use a different tech stack suitable for its task.                | <b>User Service</b> in Python, <b>Order Service</b> in Java, <b>Payment Service</b> in Node.js.                   | Freedom to choose best-fit tools and technologies. |

|                                               |                                                                                       |                                                                                               |                                                   |
|-----------------------------------------------|---------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|---------------------------------------------------|
| <b>4. Fault Isolation</b>                     | <b>A failure in one service does not crash the entire application.</b>                | <b>If Recommendation Service fails, other services like Order and Payment still work.</b>     | <b>Increased reliability and system uptime.</b>   |
| <b>5. Faster Development &amp; Deployment</b> | <b>Smaller, independent teams can develop, test, and deploy services in parallel.</b> | <b>One team works on Inventory, another on Payment, both releasing updates independently.</b> | <b>Accelerates innovation and delivery speed.</b> |
| <b>6. Easier Maintenance &amp; Debugging</b>  | <b>Each service is smaller and easier to test or fix.</b>                             | <b>Bugs in User Service can be fixed quickly without touching other modules.</b>              | <b>Simplifies debugging and reduces risk.</b>     |

# DISADVANTAGES OF MICROSERVICES ARCHITECTURE



## 1. Complexity

Managing many services and interactions can be complicated



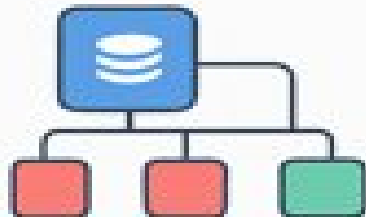
## 2. Increased Latency

Multiple inter-service calls can add latency to the system



## 3. Data Consistency

Ensuring data consistency across services can be challenging

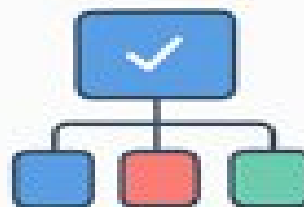


### Data Management Complexity

Since each microservice has its own database, maintaining data consistency is hard.

**Ex:** Updating order status in Order Service must also reflect in Payment Service database

**Ex:** Risk of inconsistent data of complex transactions

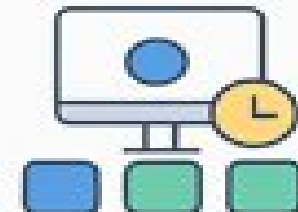


### Testing Challenges

Testing across many services is harder than testing a single monolithic application

**Ex:** End-to-end testing requires running User, Order, and Payment services together

**Ex:** More complicated integration and regression testing



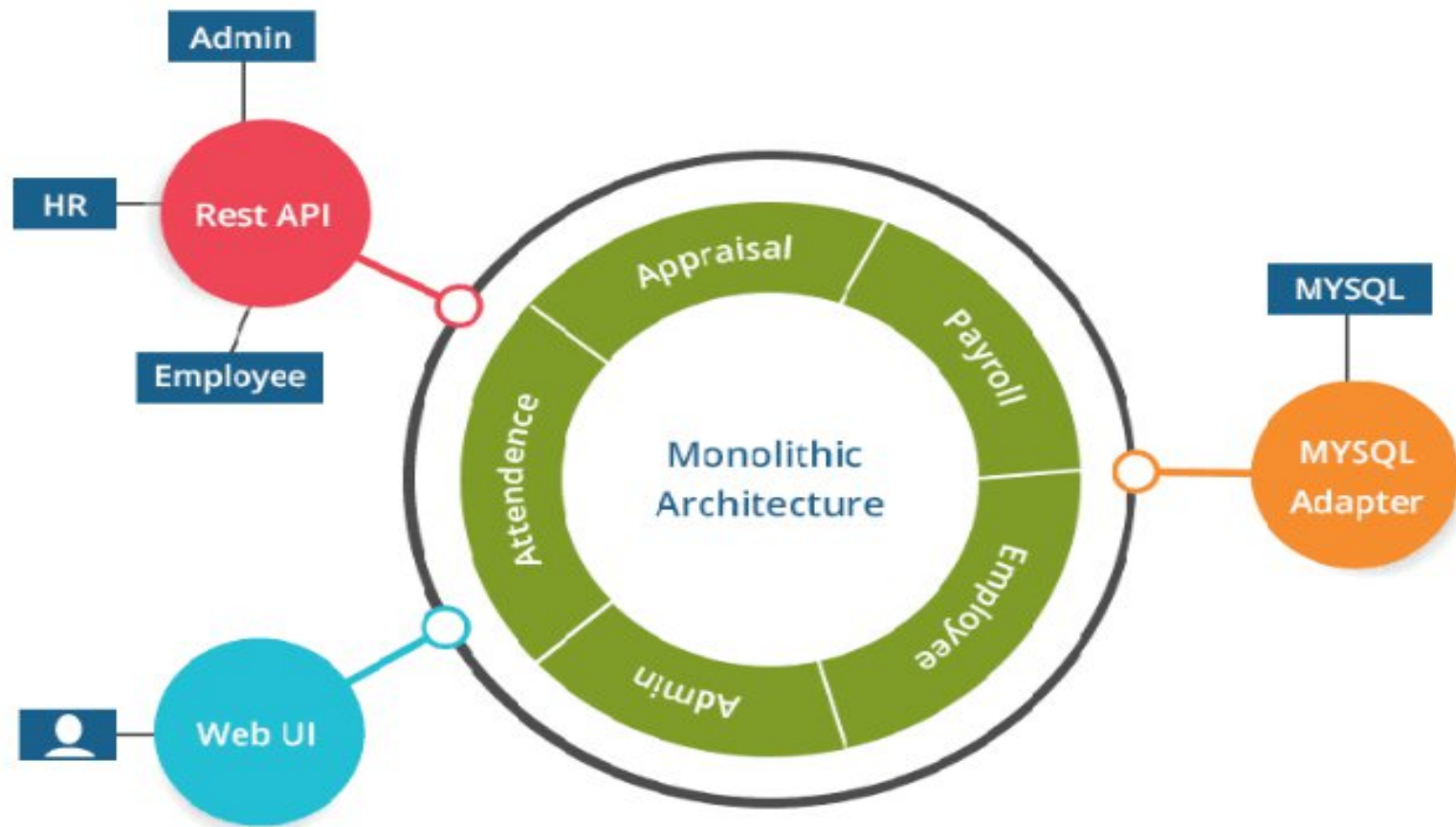
### Increased Resource Usage

Each service runs in its own container or process, consuming extra memory and CPU

**Ex:** 20 microservices running in Docker use more system resources than one monolithic app

**Ex:** Can increase infrastructure cost

# Why MicroServices? ^



- **The good thing about using microservices is that development teams are ready to rapidly build new components of apps to fulfill changing business needs.**
- **In microservices, every application resides in separate containers along with the environment it needs to run.**
- **It allows it to maximize deployment velocity and application reliability.**
- **There's no risk of interfering with the other applications. It also allows optimizing the sources micro services multiply team works on independent services.**



- **In Microservices, we run all the applications on Containers (Docker), so the boot-up time and memory consumption of applications decrease, thus the performance of the application increases.**
- **In organizations, not one or two but multiple numbers of Containers (Docker) are running.**
- **So, to manage them, we use Kubernetes. Kubernetes comes in handy because it provides various facilities like self-healing, scalability, auto-scaling, and declarative state.**

## What's the Diff: Monolithic vs Microservices

| Monolithic                                                                                                                               | Microservice                                                                            |
|------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| It is built as one large system                                                                                                          | It is built as a small independent module                                               |
| Not easy to scale based on demand                                                                                                        | Easy to scale based on demand.                                                          |
| It has a shared database                                                                                                                 | Each project and module have their own database                                         |
| Large code base makes IDE slow                                                                                                           | Each project is independent and small in size                                           |
| Continues deployment becomes difficult                                                                                                   | Continues deployment is possible here                                                   |
| It is extremely difficult to change technology or language or framework because everything is tightly coupled and depends on each other. | Easy to change technology or framework because every module and project is independent. |

# Point of Discussion



1. If an online shopping platform on AWS faces high demand for its payment service, how would scaling differ for monolithic vs. microservices architecture?
2. Can you explain what happens during a big sale event—how might microservices help Flipkart manage traffic surges differently than a monolithic system?

# Concept of Containers

- Containers are Operating System-level virtualization technologies that allow multiple isolated applications to run on a single host or instance.
- They provide agile, cost efficient, and automated application deployment in which processes are packaged with all the necessary dependencies inside their own separate instances for easy access and portability.
- Containers also offer greater levels of isolation between resources and improve resource utilization compared to traditional virtual machines by using fewer hardware resources.

- Containers are a lightweight alternative to VMs for providing **isolated** operating environments for your workloads.
- They use a different method of **abstracting** resources.
- Instead of using a hypervisor, containers share the **kernel** of the host operating system (OS).
- As a result, they avoid the infrastructure overhead of a full-blown OS and provide only those resources (i.e., **installations**, **dependencies**, and **code**) that your applications actually need.
- This means they can stop and start more quickly in response to **fluctuating scaling** requirements and offer better overall performance than VMs.

## Containers

|               |                                                                                                                                                                                                                                                                |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Communication | Use a single network for easier maintenance and configuration of multiple servers                                                                                                                                                                              |
| Scalability   | Containers can scale easily to adapt to changing workloads, and they can be deployed, replicated, and managed easily. This allows applications to adapt to changing loads with minimal effort.                                                                 |
| Management    | Container management involves organizing, scaling, and maintaining containerized applications across various environments. This includes container orchestration, which automates the deployment, networking, scaling, and lifecycle management of containers. |

## Microservices

|                                                                                                                                                                                                                                                                                        |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Modular nature allows individual clusters or services to be on separate networks with their own address spaces                                                                                                                                                                         |
| Microservices can scale independently, allowing each component of an app to be scaled depending on the resources it needs. This can efficiently use resources instead of having multiple copies of the entire application.                                                             |
| Microservices performance testing is a critical aspect of developing and maintaining microservices-based applications. This involves assessing performance metrics such as response time, throughput, and the scalability of both individual microservices and the entire application. |

# Containers and microservices



- Containers and microservices are two important technologies driving the future of development.
- While they share many similarities, there are key differences that must be understood when determining which approach is right for an application.
- Containers enable portability and resource efficiency while offering great scalability options through containerization technology such as Docker or Kubernetes.
- On the other hand, Microservices allow flexibility due to their modular design, allowing applications to scale up without major changes in architecture all the while leveraging communication between services over a network connection; this also makes DevOps easier than ever before!

- To better performance and a lower infrastructure footprint, containerized microservices are typically more robust than a traditional monolithic application.
- For example, if you deploy an entire application to a VM, when the machine goes down, the whole application goes down with it. But, with containers, you can replicate microservices across a cluster of smaller VMs.
- So, in the event of a VM failure, the application will still be able to use the other microservices in the cluster and continue to function.
-



## A Simple Container Cluster



# Microservices → Many services → Many containers



LOVELY  
PROFESSIONAL  
UNIVERSITY

## Docker Compose helps:

- ✓ Define multiple services
  - ✓ Manage networking
  - ✓ Control dependencies
  - ✓ Scale services
  - ✓ Reproduce architecture easily
- Compose = Practical enabler of microservices (especially in dev/testing)

# Docker Compose



**Docker Compose is a tool used to define and manage multi-container**

**Docker applications using a YAML configuration file.**

**Key idea:**

- ✓ **Instead of running containers individually**
- ✓ **Define the entire application stack declaratively**

## **Docker alone:**

- Runs individual containers

## **Docker Compose:**

- Manages groups of related containers

## **Its like:**

- Docker = Container-level management**

**Docker Compose = Application-level management**

# Docker Compose follows:



- ✓ **Declarative configuration**
- ✓ **Infrastructure as Code**
- ✓ **Reproducibility**
- ✓ **Automation**

# Need of Docker Compose

## Problem Without Compose

Modern applications rarely use a single container.

Typical stack:

- ✓ Web application
  - ✓ Database
  - ✓ Cache
  - ✓ Backend API
  - ✓ Message queue

Running manually:

```
docker run web  
docker run db  
docker run redis  
docker run backend
```

# Problems faces

## Too many commands

- ✗ Hard to remember parameters
- ✗ Hard to reproduce setup
- ✗ Networking complexity
- ✗ Dependency issues

# How docker solve the problem

## (A) Multi-Container Management

Define entire system in one file.

- ✓ Centralized configuration
- ✓ Single command execution

## (B) Reproducibility

Very strong academic point.

- ✓ Same environment across machines
- ✓ Same configuration for all developers
- “Works on my machine” problem → minimized.



## **(C) Simplified Networking**

### **Without Compose:**

- ✗ Manual network creation**
- ✗ IP management**

### **With Compose:**

- ✓ Automatic network creation**
- ✓ Services communicate via names**

### **Example:**

- Service db reachable at hostname db.**
- Huge conceptual advantage.**

## **(D) Dependency Management**

**Control startup order.**

- ✓ **Web depends on DB**
- ✓ **Backend depends on Cache**

**Compose manages orchestration basics.**

## **(E) Scalability**

**Scale services easily.**

- **docker compose up --scale web=3**
- **✓ No manual container duplication**

## **(F) Cleaner Configuration**

**Instead of:**

- **docker run -d -p 8080:80 --name web ...**

**You define:**

- **ports:**
  - **"8080:80"**
- **✓ Human-readable**
- **✓ Version-controlled**

# 3. Basics of Docker Compose



Now move to fundamental building blocks.

## (A) Compose File (docker-compose.yml)

Heart of Compose.

- ✓ Written in YAML
- ✓ Defines application stack

Basic structure:

```
version: "3.9"
```

```
services:
```

```
  service_name:  
    configuration
```

•Important teaching note:

•YAML = indentation-sensitive

Spacing matters

## (B) Services (Most Important Concept)



**Service = Definition / Blueprint of containers**

**NOT containers directly.**

•**Example:**

**services:**

**web:**

**image: nginx**

**Conceptually:**

- ✓ **Service defines role**
- ✓ **Container = Instance of service**

**Analogy:**

•**Class → Object**

**Service → Container**

# **(C) image vs build**

**Two ways to specify container source.**

## **Using Prebuilt Image**

- **image: nginx**
- **✓ Pull from registry**

## **Building Custom Image**

**build: .**

- ✓ **Uses Dockerfile**
- ✓ **Builds locally**

**Theory distinction:**

- ✓ **image → Existing software**
- ✓ **build → Custom software**

## (D) ports



**Defines host-container communication.**

**ports:**

**- "8080:80"**

**Explain carefully:**

**✓ HostPort : ContainerPort**

**Common student confusion → directionality.**

# **(E) volumes**



## **Persistent storage.**

- **volumes:**

- **data:/var/lib/mysql**

- **Concept:**

- ✓ **Containers ephemeral**
- ✓ **Volumes persistent**

- **Benefits:**

- ✓ **Data safety**
- ✓ **State preservation**



**Compose automatically creates:**

- ✓ **Default network**
- Conceptual advantages:**
  - ✓ **Service discovery**
  - ✓ **Built-in DNS**
  - ✓ **Name-based communication**
- Example:**
  - web → connect to db via hostname db**

# **(G) depends\_on**



**Controls startup order.**

- **depends\_on:**

- **db**

- **VERY IMPORTANT theory clarification:**

- ✓ **Ensures start order**

- ✗ **Does NOT ensure readiness**

# How Docker Compose Works

**Compose performs:**

- **Reads YAML file**
- **Pulls/builds images**
- **Creates network(s)**
- **Creates volumes**
- **Starts containers**
- **Connects services**

**Emphasize:**

**Compose automates container lifecycle management.**

# Docker Compose vs Docker CLI



LOVELY  
PROFESSIONAL  
UNIVERSITY

## Docker CLI

Manual commands

Individual containers

Hard to reproduce

Verbose

## Docker Compose

Declarative file

Multi-container apps

Easily reproducible

Clean & structured

# Basic Commands

## Initialization & Setup

- **docker-compose version**  
→ Check installed version
- **docker-compose config**  
→ Validate YAML file syntax

Create the file : notepad docker-compose.yml

version: "3.9"

services:

web:

image: nginx

ports:

- "8080:80"



## Starting Services

- **docker-compose up**  
→ **Start all services (foreground)**
- **docker-compose up -d**  
→ **Start in detached mode (background)**
- **docker-compose up --build**  
→ **Rebuild images before starting**



## Stopping Services

- **docker-compose down**  
→ Stop & remove containers, networks
- **docker-compose stop**  
→ Stop containers (no removal)
- **docker-compose start**  
→ Restart stopped containers

## Monitoring & Debugging

- **docker-compose ps**  
→ List running services
- **docker-compose logs**  
→ View logs
- **docker-compose logs -f**  
→ Live logs

- **Executing Commands**
- **docker-compose exec <service> bash**  
→ Enter container shell
- **docker-compose up --scale web=3**  
→ Run multiple instances



**A company wants to deploy a 3-tier web application using Docker Compose:**

- **Frontend → React (Node.js)**
- **Backend → Spring Boot API**
- **Database → PostgreSQL**
- **Requirements:**

**All services must communicate**

- **Environment variables should be used**
- **Data should persist using volumes**
- **Services must start in dependency order**

- **Create docker-compose.yaml file**
- **docker compose up -d**
- **docker compose config --services**
  
- **docker exec -it postgres-db psql -U postgres- check database**

- **Create a Docker Compose setup for a Node.js application connected to MongoDB.**

**Requirements:**

- **Node.js app runs on port 3000**
- **MongoDB runs on default port 27017**
- **Use environment variables for DB connection**
- **Ensure MongoDB starts before Node.js**



- **version: '3.8'**
- **services:**
- **app:**
- **image: node:18**
- **container\_name: node-app**
- **working\_dir: /usr/src/app**
- **volumes:**
- **- ./usr/src/app**
- **command: npm start**
- **ports:**
- **- "3000:3000"**
- **environment:**
- **- MONGO\_URL=mongodb://mongo:27017/mydb**
- **depends\_on:**
- **- mongo**
- **mongo:**
- **image: mongo:6**
- **container\_name: mongo-db**
- **ports:**
- **- "27017:27017"**
- **volumes:**
- **- mongo-data:/data/db**

- **docker compose up -d**
- **docker compose ps**
- **docker compose logs app**
- **docker compose down**

- **Deploy a WordPress website using Docker Compose with MySQL as the database.**  
**Requirements:**
- **WordPress runs on port 8080**
- **MySQL uses persistent storage**
- **Use environment variables for DB configuration**
- **Ensure proper service dependency**



- **version: '3.8'**

- **services:**

- **wordpress:**

- **image:** wordpress:latest

- **container\_name:** wordpress-app

- **ports:**

- **- "8080:80"**

- **environment:**

- **WORDPRESS\_DB\_HOST:** db:3306

- **WORDPRESS\_DB\_USER:** wpuser

- **WORDPRESS\_DB\_PASSWORD:** wppass

- **WORDPRESS\_DB\_NAME:** wpdb

- **depends\_on:**

- **- db**

- **db:**

- **image:** mysql:5.7

- **container\_name:** mysql-db

- **restart:** always

- **environment:**

- **MYSQL\_DATABASE:** wpdb

- **MYSQL\_USER:** wpuser

- **MYSQL\_PASSWORD:** wppass

# MAVEN

## BUILD AUTOMATION

---

*Lifecycle • Dependency Management • Plugins • Docker Integration*



# Why Build Tools Exist

*The problems that triggered automation*

## Manual Compilation Pain

Developers had to remember exact javac commands, classpaths, and dependencies for every build — error-prone and time-consuming.

## Dependency Hell

Managing JAR files manually led to version conflicts, missing libraries, and the infamous 'works on my machine' syndrome.

## Inconsistent Builds

Without a standard process, builds differed between developers, CI servers, and production — causing unpredictable failures.

## Repetitive Tasks

Testing, packaging, and deploying were manual steps that had to be repeated identically every release cycle.

*Maven solves all of the above through a standardized, declarative build system.*

# Project Object Model (POM)

*The heart of every Maven project — pom.xml*

**GroupId:** Unique org identifier e.g. com.example

**ArtifactId:** Project/module name e.g. my-app

**Version:** Release version e.g. 1.0.0-SNAPSHOT

**Packaging:** Output format: jar, war, pom

**Dependencies:** All library declarations live here

**Build:** Plugin config, source dirs, output

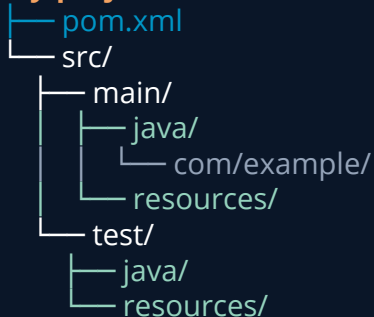
**Profiles:** Environment-specific configurations

```
<project>
  <groupId>com.example</groupId>
  <artifactId>my-app</artifactId>
  <version>1.0.0-SNAPSHOT</version>
  <packaging>jar</packaging>
  <dependencies>
    <dependency>
      ...
    </dependency>
  </dependencies>
</project>
```

# Standard Maven Directory Structure

Convention over configuration

## my-project/



*target/ ← generated output*

## src/main/java

Production Java source code. Maven compiles everything here.

## src/main/resources

Config files, properties, XMLs bundled into the artifact.

## src/test/java

Unit and integration test classes (not bundled in output).

## src/test/resources

Test-only config files and fixtures.

## target/

Auto-generated build output — classes, JARs, WAR files.

## pom.xml

Project descriptor — the single source of truth for the build.

# Build Lifecycle Phases

*Maven's three built-in lifecycles: default, clean, site*

## 1 validate

Checks project is correct & all required info is available

## 2 compile

Compiles source code of the project into .class files

## 3 test

Runs unit tests using a testing framework (e.g., JUnit)

## 4 package

Bundles compiled code into distributable format (JAR/WAR)

## 5 verify

Runs integration checks to ensure quality criteria are met

## 6 install

Installs artifact into local Maven repository (~/.m2)

## 7 deploy

Copies final artifact to remote repository for sharing

*Each phase triggers all previous phases automatically*

# Parent POM

Centralizing configuration across multi-module projects

## What is a Parent POM?

A Parent POM is a special POM that other projects inherit from. Child modules automatically receive all dependency versions, plugin configs, and properties defined in the parent, ensuring DRY (Don't Repeat Yourself) principles across large projects.

### Key Benefits:

- Centralized dependency version management
- Shared plugin configuration
- Consistent build behavior
- Single place to update versions
- Spring Boot provides its own parent POM

```
<!-- Parent POM -->  
<parent>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-parent</artifactId>  
  <version>3.2.0</version>  
  <relativePath/>  
</parent>  
  
<!-- Child inherits all configs -->  
<!-- No need to repeat versions -->
```

# Dependency Scope

*Controlling when and where dependencies are available*

| Scope    | Compile | Test | Runtime | Description                                                                                     |
|----------|---------|------|---------|-------------------------------------------------------------------------------------------------|
| compile  | ✓       | ✓    | ✓       | Default scope. Available everywhere — compile, test, and runtime classpaths.                    |
| provided | ✓       | ✓    | ✗       | Available at compile/test but NOT bundled. JEE containers (e.g., Tomcat) provide it at runtime. |
| runtime  | ✗       | ✓    | ✓       | Not needed to compile but needed at runtime. JDBC drivers are a typical example.                |
| test     | ✗       | ✓    | ✗       | Only for test compilation and execution. JUnit, Mockito are declared test scope.                |
| system   | ✓       | ✓    | ✗       | Like provided but references a JAR via explicit path. Discouraged — not portable.               |
| import   | —       | —    | —       | Only for <dependencyManagement>. Imports a BOM (Bill of Materials) from another POM.            |

# Transitive Dependencies & Version Conflicts

How Maven resolves the dependency graph

## Transitive Dependencies

When you declare  $A \rightarrow B$ , Maven automatically pulls in all of B's dependencies ( $B \rightarrow C$ ,  $C \rightarrow D \dots$ ). You get the full tree without manually declaring each transitive dep.

## Conflict Resolution — Nearest Wins

If two paths lead to the same artifact with different versions, Maven picks the version closest to the project root in the dependency tree (shortest path).

## Solutions:

- Explicit direct dependency declaration
- `<exclusions>` to remove unwanted transitives
- `<dependencyManagement>` to enforce a version
- BOM (Bill of Materials) import

## Excluding a Transitive Dep:

```
<dependency>
  <groupId>com.example</groupId>
  <artifactId>lib-a</artifactId>
  <exclusions>
    <exclusion>
      <groupId>com.bad</groupId>
      <artifactId>old-lib</artifactId>
    </exclusion>
  </exclusions>
</dependency>
```

```
<!-- Force version via mgmt: -->
<dependencyManagement>
  <dependencies>
    <!-- pin specific version -->
  </dependencies>
</dependencyManagement>
```

# Maven Plugins & Execution

*Plugins are how Maven does everything*

Plugins bind goals to lifecycle phases. A goal is a specific task (like `compile:compile` or `surefire:test`). Each plugin has one or more goals.

## Compiler Plugin

Controls Java source/target version for compilation.

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <configuration>
    <source>17</source>
    <target>17</target>
  </configuration>
</plugin>
```

## Surefire Plugin

Runs unit tests. Generates test reports. Skips with `-DskipTests`.

```
<plugin>
  <artifactId>maven-surefire-plugin</artifactId>
  <version>3.1.2</version>
  <configuration>
    <includes>**/*Test.java</includes>
  </configuration>
</plugin>
```

`mvn compile` | `mvn test` | `mvn package` | `mvn verify` | `mvn deploy`



# Shade Plugin — Creating an Uber JAR

*Packaging all dependencies into a single executable JAR*

## What is an Uber JAR?

An Uber (German for 'over/super') JAR is a single fat JAR that contains your compiled code PLUS all its dependency JARs merged inside it.

This makes deployment trivially simple — ship one file, run anywhere with `java -jar`.

## When to use:

- CLI tools that need to be self-contained
- Microservices deployed to containers
- Spark/Hadoop jobs submitted to clusters
- Anywhere classpath management is painful

⚠ *Watch for class name conflicts from merged JARs*

```
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-shade-plugin</artifactId>
  <version>3.5.0</version>
  <executions>
    <execution>
      <phase>package</phase>
      <goals><goal>shade</goal></goals>
      <configuration>
        <transformers>
          <transformer
            impl="ManifestResourceTransformer">
            <mainClass>com.App</mainClass>
          </transformer>
        </transformers>
      </configuration>
    </execution></executions>
</plugin>
```

# Maven Wrapper (mvnw)

*Ensuring consistent Maven version across all environments*

## The Problem

Different developers and CI servers may have different Maven versions installed, causing 'it works on my machine' build failures.

## The Solution

mvnw (Maven Wrapper) is a script committed to your repo. It auto-downloads and uses the exact Maven version specified in `.mvn/wrapper/maven-wrapper.properties`.

## How to Generate

Run: `mvn wrapper:wrapper`  
This creates `mvnw`, `mvnw.cmd`, and `.mvn/` directory in your project root.

## Usage

```
./mvnw clean package
./mvnw spring-boot:run
# Windows: mvnw.cmd clean install
```

✓ No Maven pre-installation required   ✓ Version pinned in repo   ✓ Works offline (cached in `~/.m2/wrapper`)

# Maven & Docker Integration

*dockerfile-maven-plugin, Dockerizing apps, Pushing to registries*

## Integration Overview

- 1. Build JAR**  
mvn package produces the application JAR in target/
- 2. Write Dockerfile**  
Multi-stage Dockerfile: base image → copy JAR → ENTRYPOINT
- 3. Build Image**  
dockerfile-maven-plugin runs docker build as part of mvn install
- 4. Tag Image**  
Plugin tags image with project version automatically
- 5. Push to Registry**  
mvn deploy pushes image to Docker Hub / ECR / GCR

## Dockerfile (multi-stage):

```
FROM eclipse-temurin:17-jdk AS build
WORKDIR /app
COPY . .
RUN ./mvnw package -DskipTests
```

```
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

```
# Build & push via Maven:
mvn dockerfile:build dockerfile:push
```

## **Pull Maven Image (One-Time)**

```
docker pull maven:3.9.10-eclipse-temurin-17
```

```
docker run --rm -it -v ${PWD}:/app -w /app maven:3.9.10-eclipse-temurin-17
```

```
mvn archetype:generate -DgroupId=com.example -DartifactId=demo-app -  
DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

```
mkdir test123
```

```
cd test123
```

```
docker run --rm -it -v %cd%:/app -w /app maven:3.9.10-eclipse-temurin-17 mvn  
archetype:generate -DgroupId=com.example -DartifactId=demo-app -  
DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

```
cd demo-app
```

```
docker run --rm -it -v %cd%:/app -w /app maven:3.9.10-eclipse-temurin-17 mvn clean  
install
```



## INT 332 – Devops Virtualization And Configuration Management

# Introduction to GitHub Actions



**GitHub Actions is a CI/CD (Continuous Integration & Continuous Deployment) platform that allows you to automate workflows directly inside your GitHub repository.**

**It helps in:**

- **Building code**
- **Running tests**
- **Deploying applications**
- **Automating repetitive development tasks**

**Analogy:**

**Think of GitHub Actions like a smart assistant that automatically performs tasks (like testing or deployment) whenever something happens in your repository.**



# Understanding Workflow Automation



## What is a Workflow?

- A workflow is an automated process defined in a YAML file.
- Stored inside `.github/workflows/`
- Contains one or more jobs
- Each job has steps (commands/actions)





# Basic Structure of a Workflow

**name:** My First Workflow

**on:** push

**jobs:**

**build:**

**runs-on:** ubuntu-latest

**steps:**

- **name:** Checkout code

**uses:** actions/checkout@v3

- **name:** Run a script

**run:** echo "Hello World"

# Components of Workflow



- **Event (on) → When to run**
- **Jobs → What to do**
- **Steps → How to do**
- **Actions → Predefined reusable tasks**

# Events in GitHub Actions

## What are Events?

- **Events are activities that trigger workflows.**

| Event             | Description                   |
|-------------------|-------------------------------|
| push              | When code is pushed           |
| pull_request      | When PR is created/updated    |
| workflow_dispatch | Manual trigger                |
| schedule          | Runs on cron timing           |
| issues            | When issue is created/updated |

# Triggers in GitHub Actions

## What is a Trigger?

A trigger is the condition under which an event runs a workflow.

- Event = "push"  
Trigger = "only on main branch"

## Example with Filters

on:

push:

branches:

- main

paths:

- - 'src/\*\*'

# Types of Triggers

## 1. Branch-based Trigger

branches: [main, dev]

## 2. Path-based Trigger

paths: - 'app/\*\*'

## 3. Tag-based Trigger

tags:- 'v1.\*'



## 4. Time-based Trigger (Cron)

on:

schedule:

- cron: '0 0 \* \* \*'

## 5. Workflow Directory Structure

repository/

```
├── .github/
│   └── workflows/
│       ├── build.yml
│       └── deploy.yml
│
├── src/
├── tests/
└── README.md
```



## 6. Jobs and Steps

### Jobs

A workflow contains multiple jobs

Jobs run in parallel by default

jobs:

build:

test:



## Steps

- Each job has steps executed sequentially

### steps:

- name: Step 1

run: echo "Hello"



## 7. Actions (Reusable Components)

**What are Actions?**

**Reusable units of code.**

**Example:**

**uses: actions/checkout@v3**

## 8. Runners

**What is a Runner?**

**A server that runs your workflow.**

**Types:**

- **GitHub-hosted (Ubuntu, Windows, macOS)**
- **Self-hosted**

# Real-Life Example Workflow



LOVELY  
PROFESSIONAL  
UNIVERSITY

**name: Java CI Pipeline**

- **on:**
- **push:**
- **branches: [main]**
- **jobs:**
- **build:**
- **runs-on: ubuntu-latest**
- **steps:**
  - **- uses: actions/checkout@v3**
  - **- name: Set up Java**
  - **uses: actions/setup-java@v3**
  - **with:**
  - **java-version: '17'**
  - **- name: Build with Maven**
  - **run: mvn clean install**



LOVELY  
PROFESSIONAL  
UNIVERSITY

# GitHub Actions Workflows

# 1. Workflow Triggers in GitHub Actions



- **1.1 Push Trigger**

**Runs workflow when code is pushed.**

**on:**

**push:**

**branches: [main]**

**Use case:**

- **Continuous Integration (CI)**
- **Build & test on every commit**

## 1.2 Pull Request Trigger



- Triggered when a PR is opened, updated, or merged.

on:

**pull\_request:**

**branches: [main]**

**Use case:**

- Code review validation
- Testing before merging

## 1.3 Schedule Trigger (Cron Jobs)

**Runs workflow at scheduled intervals.**

**on:**

**schedule:**

**- cron: '0 0 \* \* \*'**

**Example:**

- **Nightly builds**
- **Automated backups**

## 1.4 Manual Trigger (workflow\_dispatch)

**Allows user to trigger workflow manually from GitHub UI.**

**on:**

**workflow\_dispatch:**

**Use case:**

- **On-demand deployments**
- **Testing pipelines manually**

## 2.Jobs & Matrix Strategies

**What is a Job?**

**A job is a set of steps executed on the same runner.**

**jobs:**

**build:**

**runs-on: ubuntu-latest**



## Matrix Strategy (Parallel Execution)

- Runs the same job with multiple configurations.

jobs:

test:

runs-on: ubuntu-latest

strategy:

matrix:

node-version: [14, 16, 18]

steps:

- uses: actions/setup-node@v3

with:

node-version: \${matrix.node-version}

### 3. Steps & Shell Commands

#### Steps

- Each job consists of sequential steps.

#### Steps:

- - name: Step 1
- run: echo "Hello World"

# Shell Commands



- **Run OS-level commands directly.**

**- name: List files**

**run: ls -la**

# Quick Quiz



LOVELY  
PROFESSIONAL  
UNIVERSITY

**1. Identify the correct order of execution:**

- **A. Steps → Jobs → Workflow**
- B. Workflow → Jobs → Steps**
- C. Jobs → Workflow → Steps**
- D. Steps → Workflow → Jobs**

**2. Which of the following is a reusable unit in GitHub Actions?**

- A. Job**
- B. Step**
- C. Action**
- D. Runner**



**3. A workflow in GitHub Actions is defined using:**

- **A. JSON file**
- B. YAML file**
- C. XML file**
- D. Properties file**

**4. Where are workflow files stored in a repository?**

- **A. /src/workflows/**
- B. /github/actions/**
- C. /.github/workflows/**
- D. /ci-cd/**



5. Which component defines *when* a workflow runs?

- A. Jobs
- B. Steps
- C. Events
- D. Actions



**6. Which of the following is a valid push trigger?**

**A.**

- **on: push**

**B.**

- **on:  
  push:  
    branches: [main]**

**C.**

- **trigger: push**

**D.**

- **event: push**



**7. What is the role of a runner?**

- A. Stores code**
- B. Executes workflows**
- C. Triggers events**
- D. Builds UI**

**8. Jobs in a workflow run:**

- A. Sequentially by default**
- B. In parallel by default**
- C. Only manually**
- D. Only on Windows**



## **9. Steps inside a job run:**

- **A. In parallel**  
**B. Randomly**  
**C. Sequentially**  
**D. Only once**

**10. Identify the correct order of execution:**

- A. Steps → Jobs → Workflow**
- B. Workflow → Jobs → Steps**
- C. Jobs → Workflow → Steps**
- D. Steps → Workflow → Jobs**



**11. What does this cron expression mean?**

**cron: '0 0 \* \* \*'**

- A. Every hour**
- B. Every minute**
- C. Every day at midnight**
- D. Every week**



**12. You want to test code on multiple Node.js versions simultaneously. What should you use?**

- A. Steps**
- B. Matrix strategy**
- C. Runner**
- D. Trigger**



**13. If a workflow fails at step 2, what happens?**

- A. Step 3 still runs**
- B. Job stops execution**
- C. Workflow restarts**
- D. Next job starts**



## 14. GitHub-hosted runners support:

- A. Only Linux
- B. Only Windows
- C. Linux, Windows, macOS
- D. None

## Practice Questions:

### 1. Identify Error

- Find the error in the following YAML:

**name: CI Pipeline**

**on push:**

**branches:**

**- main**

**jobs:**

**build:**

**runs-on ubuntu-latest**

**steps:**

**- name Checkout code**

**uses actions/checkout@v3**

2.

## Workflow Trigger

➤ Write a GitHub Actions workflow that triggers:

On push

Only for develop branch



### 3. Basic Workflow File



➤ Create a minimal GitHub Actions .yml file that:

Checks out code

Prints "Hello World"

## 4. Complete CI Workflow

- Write a full .yml file that:
  - Triggers on push
  - Runs on Ubuntu
  - Steps:
    - Checkout code
    - Build project (use echo "Building")
    - Test project (use echo "Testing")

## 5. Multiple Steps

➤ Write a job with two steps:

- Install dependencies
- Run tests

6.



LOVELY  
PROFESSIONAL  
UNIVERSITY

**jobs:**

**build:**

**runs-on: ubuntu-latest**

**steps:**

**- name: Build**

**run: ls**

7.

- **name: Node CI**
- **on:**
  - **push:**
  - **branches:**
    - **- main**
- **jobs:**
- **build:**
  - **runs-on: ubuntu-latest**
  - **steps:**
    - **- name: Install dependencies**
    - **run: npm install**
    - **- name: Run tests**
    - **run: npm test**



**L** OVELY  
**P** ROFESSIONAL  
**U** NIVERSITY

# CI /CD

CI/CD is a process of automating code integration, build creation, and testing.

- GitLab CI/CD is a popular tool for implementing CI/CD pipelines.
- It involves steps like code commit, pull request, code merge, build creation, and test case execution.

- Tools like Selenium and JUnit are used for uploading test cases.
- Tools like BlackTurk and SonarQube are used for security vulnerability checks and static code analysis.
- Builds are uploaded to a build repository like SonarTypeNexus and JFrog Artifactory.
- Integration is done using tools like Jenkins, AWS CodePipeline, and GitLab CI.



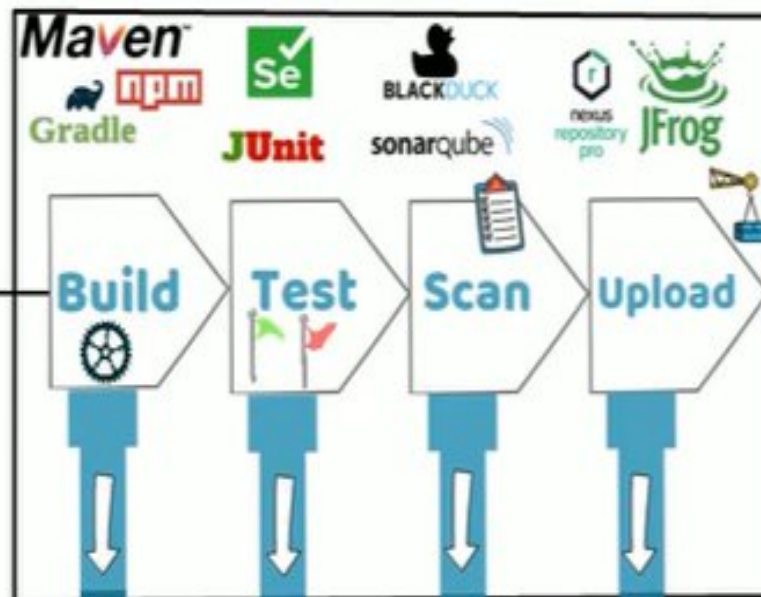
- Deployment is triggered for the dev server first and then for the UAD server after passing sanity checks.
- Fraud deployment is triggered after passing the sanity check on the UAD server.

- Continuous delivery involves continuous feedback and monitoring at all stages of the deployment lifecycle.
- A release cycle includes build, test, scan, upload, development deployment, UAT deployment, and production deployment.

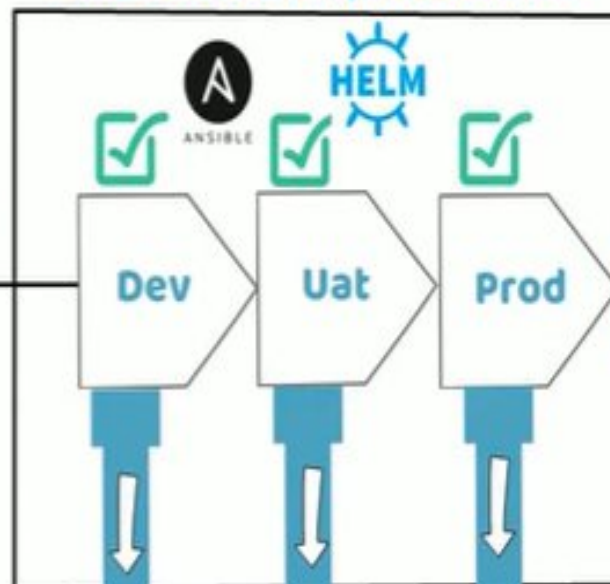


AWS CodePipeline

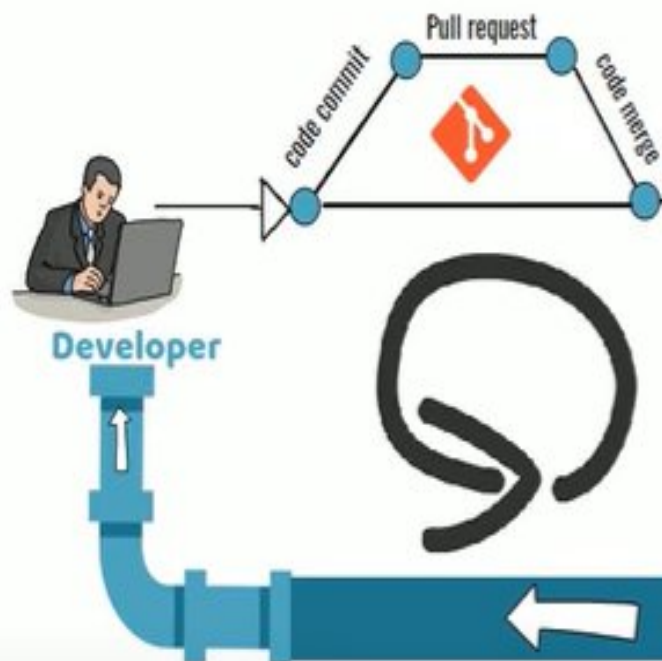
## COUNTINUOUS INTEGRATION



## COUNTINUOUS DEPLOYMENT/DELIVERY



## CONTINUOUS FEEDBACK AND MONITORING



## **What is Jenkins?**

Jenkins is an open-source automation server.

It helps automate the parts of software development related to building, testing, and deploying, facilitating Continuous Integration and Continuous Delivery (CI/CD).

## Why Use Jenkins?

- Automates code builds and tests
- Integrates with Git, Docker, Maven, Kubernetes, etc.
- Supports pipelines for end-to-end automation
- Has 1800+ plugins for customization
- Works across different OS platforms

| Term        | Description                                                                     |
|-------------|---------------------------------------------------------------------------------|
| Job/Project | A task to be executed (e.g., build a project)                                   |
| Build       | A single run of a job                                                           |
| Node        | A machine where Jenkins runs jobs (Master or Agent/Slave)                       |
| Pipeline    | A series of steps (scripted or declarative) to define CI/CD workflow            |
| Plugin      | Add-on that gives Jenkins extra functionality (e.g., Git plugin, Docker plugin) |

# Jenkins Setup Basics

## 1.Install Jenkins

- Download from [jenkins.io](https://jenkins.io)
- Run with Java (java -jar jenkins.war) or install via Docker

## 2.Access Jenkins Dashboard

- Go to <http://localhost:8080> in your browser
- Enter the admin password (found in the Jenkins home folder)

## **Install Suggested Plugins**

- **During setup, Jenkins will recommend a few plugins—install them.**

## **Create First Admin User**

- **Set up username, password, and email.**



# Creating Your First Job

- Click “New Item”
- Enter a name
- Choose “Freestyle project”
- Configure:
  - Source Code Management (Git)
  - Build Trigger (e.g., Poll SCM or on Push)
  - Build (e.g., Execute Shell or Invoke Maven)
- Click “Save” and then “Build Now”

## Think of a Job as:

"Tell Jenkins what to do and when to do it."

Jobs are the core unit of automation in Jenkins.

- Automating Repetitive Work
- Enabling Continuous Integration (CI)
- Setting Up Continuous Delivery (CD)
- Custom Workflows and Pipelines
- Integrations

End-to-End CI/CD Pipeline  
Implementation using Jenkins,  
GitHub, Maven, and Docker  
Add Required Plugins

|                                   |            |
|-----------------------------------|------------|
| Pipeline Graph Analysis           | Success    |
| Pipeline: REST API                | Success    |
| Pipeline: Stage View              | Success    |
| Dev Tools Symbols API             | Success    |
| Javadoc                           | Success    |
| JSch dependency                   | Installing |
| Maven Integration                 | Pending    |
| Authentication Tokens API         | Pending    |
| Docker Commons                    | Pending    |
| SSH server                        | Pending    |
| Git server                        | Pending    |
| Docker Pipeline                   | Pending    |
| HTML Publisher                    | Pending    |
| Role-based Authorization Strategy | Pending    |
| SSH Agent                         | Pending    |
| Commons Compress API              | Pending    |
| Pipeline Utility Steps            | Pending    |
| Loading plugin extensions         | Pending    |

```
mkdir neww
cd new
mkdir -p src/main/java/com/example
notepad src/main/java/com/example/Application.java
package com.example;
```

```
import org.springframework.boot.*;
import org.springframework.boot.autoconfigure.*;
import org.springframework.web.bind.annotation.*;
```

```
@SpringBootApplication
@RestController
public class Application {

    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }

    @GetMapping("/")
    public String home() {
        return "CI/CD Working!";
    }
}
```

## **notepad Dockerfile**

FROM openjdk:17

COPY target/\*.jar app.jar

ENTRYPOINT ["java","-jar","/app.jar"]

## **notepad Jenkinsfile**

```
pipeline {
```

```
    agent any
```

```
    tools {
```

```
        maven 'Maven'
```

```
    }
```

```
    stages {
```

```
        stage('Build') {
```

```
            steps {
```

```
                bat 'mvn clean package'
```

```
            }
```

```
        }
```

```
    }
```

```
}
```

## Create pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>app</artifactId>
  <version>1.0</version>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.0</version>
  </parent>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
```

git init

git add .

git commit -m "added project files"

git branch -M main

git remote add origin <https://github.com/preetkaur18/newww.git>

git push -u origin main

## **VERIFY ON GITHUB**

Jenkinsfile

pom.xml

Dockerfile

src folder



Apply restart with Jenkins

- Go to **Jenkins Dashboard**
- ci-cd-project
- Click on **Configure**
- Scroll down in configuration page
- Find the **Pipeline Section**
- Set **Definition** as:  
Pipeline script from SCM
- Set **SCM** as:  
Git
- Enter Repository URL:  
<https://github.com/preetkaur18/newww.git>
- Enter Branch:  
\*/main
- Enter Script Path:  
Jenkinsfile
- Click **Save**
- **Apply build now**

# What is a Jenkins Pipeline?

A pipeline is a set of automated steps that Jenkins follows to build, test, and deploy your application. Think of it as a scripted flowchart for your DevOps process.

# Why Use a Pipeline?

- Automate the entire CI/CD process
- Make builds repeatable and version-controlled
- Run complex workflows (parallel stages, conditional logic, etc.)
- Use code to define build steps (called "Pipeline as Code")

# Two Types of Pipelines

Type

Description

Declarative

Easier to read and write.  
Structured and recommended  
for beginners.

Scripted

More flexible but uses Groovy  
scripting, best for advanced  
users.

# What Are Plugins in Jenkins?

Plugins are add-ons that extend Jenkins' core functionality.  
They help Jenkins integrate with tools like:

- GitHub, GitLab
- Docker
- Slack
- Maven, Gradle
- JIRA
- Kubernetes
- And much more!

Basically, if Jenkins needs to do anything beyond basic build automation, there's probably a plugin for it.

# Why Are Plugins Important?

Without plugins, Jenkins is just a basic automation server. Plugins let you:

- Connect to source code repositories (e.g., Git plugin)
- Add build tools (e.g., Maven, Gradle plugin)
- Send notifications (e.g., Email, Slack)
- Use containers (e.g., Docker plugin)
- Define complex pipelines (Pipeline plugin)

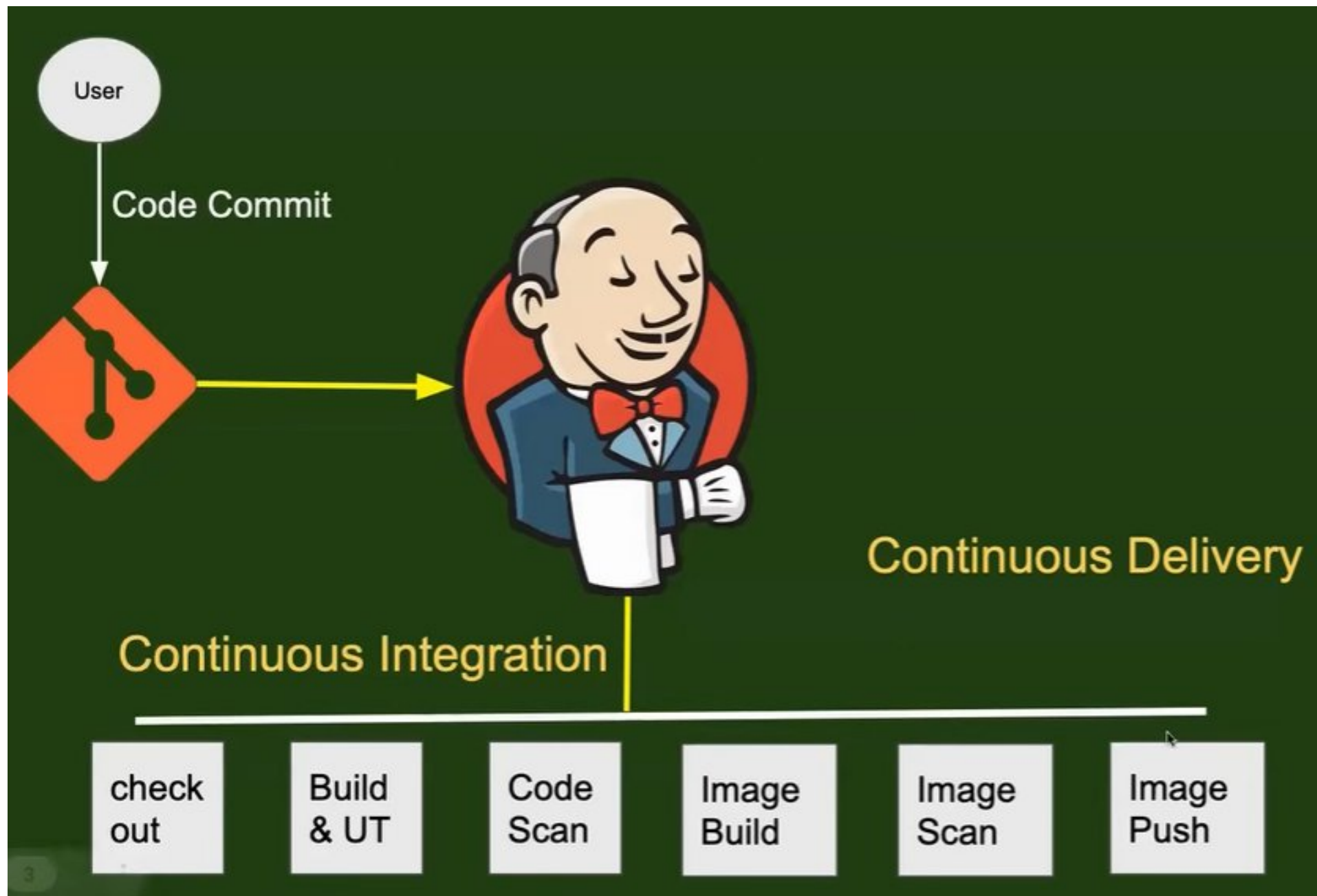
| Plugin Name        | What It Does                                            |
|--------------------|---------------------------------------------------------|
| Git Plugin         | Lets Jenkins pull code from Git                         |
| Pipeline Plugin    | Enables pipeline-as-code using <code>Jenkinsfile</code> |
| Blue Ocean         | A modern UI for Jenkins Pipelines                       |
| Docker Pipeline    | Lets you run Docker containers inside pipelines         |
| Slack Notification | Sends build status messages to Slack                    |
| Email Extension    | Sends custom emails on build success/failure            |
| JUnit Plugin       | Publishes test results                                  |
| Credentials Plugin | Securely stores passwords, tokens, and SSH keys         |

# How to Install a Plugin

- Go to Jenkins Dashboard
- Click on Manage Jenkins
- Click on Manage Plugins
- Search in the Available tab
- Click Install without restart or Download now and install after restart

Too many plugins = performance issues or conflicts. Only install what you need.





# Overview of Jenkins

- Jenkins is an open-source automation server designed to assist developers in automating the software development lifecycle.
- Initially created for automating the build process for Java, Jenkins now supports a variety of programming languages and technologies.
- By automating repetitive tasks—such as building, testing, and deployment—Jenkins allows developers to concentrate on coding.

## **Key Features of Jenkins**

- Continuous Integration (CI) and Continuous Delivery (CD) are core features, enabling automatic building, testing, and deployment of code changes.
- Jenkins utilizes a rich ecosystem of plugins, which extend its functionality and support automation for diverse tasks in the development workflow.
- Jenkins pipelines allow developers to define the entire build, test, and deployment process as code, facilitating version control and collaboration.

## How Jenkins Enhances Workflow

- By automating routine processes, Jenkins reduces the potential for human error and saves time, streamlining the software development process.
- Integrations with other tools in the software development ecosystem make Jenkins adaptable to existing workflows, enhancing productivity.
- Developers can write pipeline scripts using Groovy, a domain-specific language, enabling the definition of complex workflows and task automation.

## **Importance for Developers and DevOps Engineers**

- Jenkins is considered an essential tool in the software development field, providing significant benefits in CI/CD capabilities.
- The automation features of Jenkins enable teams to catch errors and bugs early in the development cycle, improving software quality.
- With its robust automation capabilities and wide-ranging integrations, Jenkins empowers developers to focus on creating high-quality code.

Dashboard ▾

 New Item

 People

 Build History

 Manage Jenkins

 My Views

 Lockable Resources

 New View

Build Queue

^

No builds in the queue.

Build Executor Status

^

1 Idle

2 Idle

[add description](#)

# Welcome to Jenkins!

This page is where your Jenkins jobs will be displayed. To get started, you can set up distributed builds or start building a software project.

## Start building your software project

Create a job →

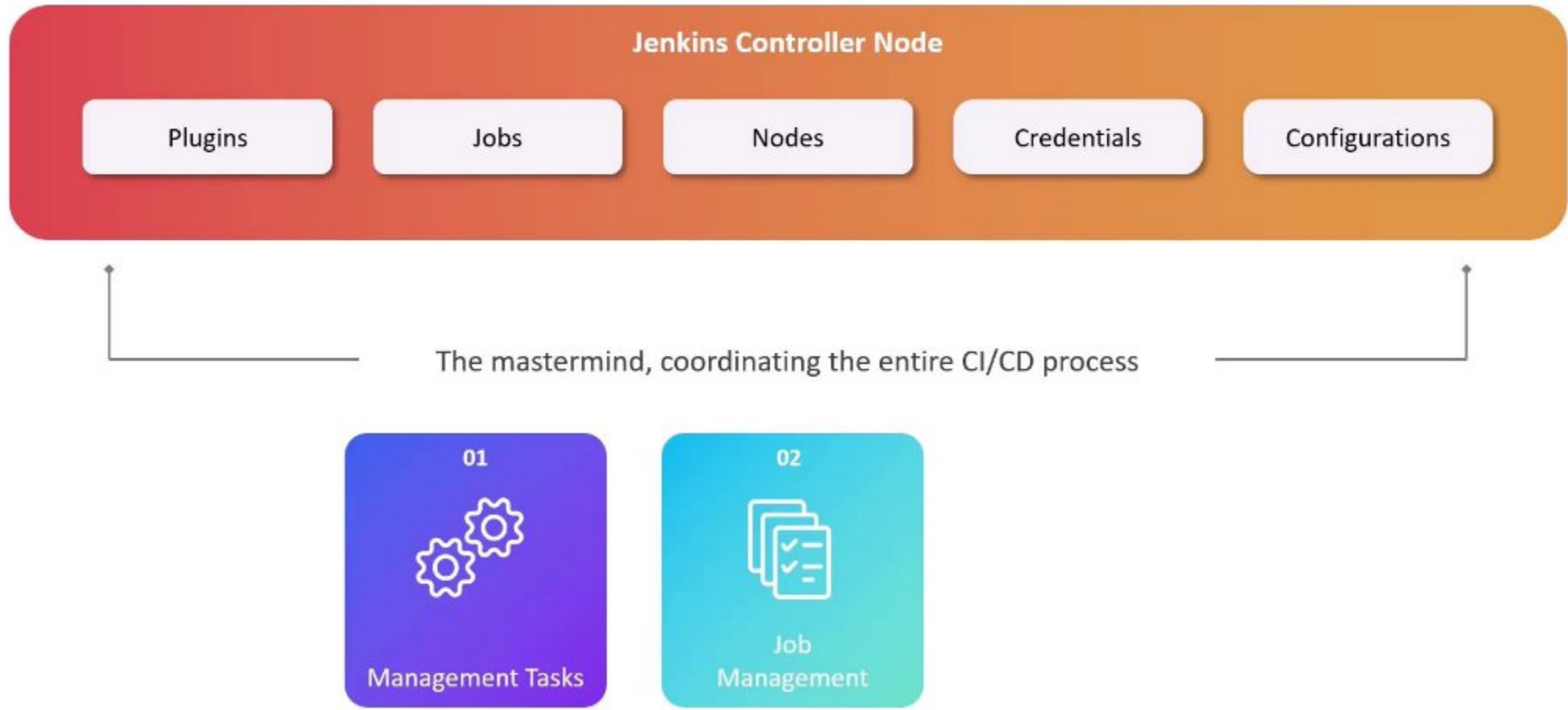
## Set up a distributed build

Set up an agent →

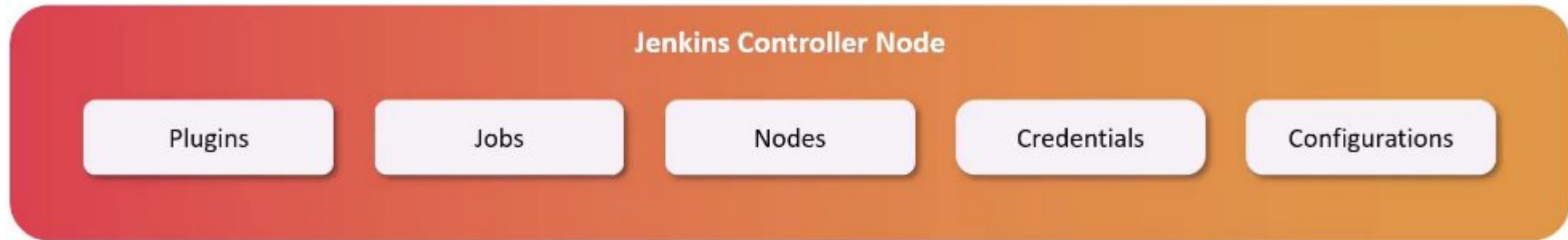
Configure a cloud →

Learn more about distributed builds ↗

## Jenkins Architecture



## Jenkins Architecture



01

### Basic Deployment

Controller and worker node are the same



Limited to  
smaller project

02

### Advanced Deployment

Separate controller and worker nodes



Prevents  
Accidental  
Damage



Boosts  
Performance



Scalability for  
Growth



## Jenkins Architecture

